

Pure Borrow: Linear Haskell Meets Rust-Style Borrowing*

YUSUKE MATSUSHITA, Kyoto University, Japan

HIROMI ISHII, JIJ Inc., Japan

A promising approach to unifying functional and imperative programming paradigms is to localize mutation using linear or affine types. Haskell, a purely functional language, was recently extended with linear types by Bernardy et al., in the name of Linear Haskell. However, it remained unknown whether such a pure language could safely support non-local *borrowing* in the style of Rust, where each borrower can be freely split and dropped without direct communication of ownership back to the lender.

We answer this question affirmatively with *Pure Borrow*, a novel framework that realizes Rust-style borrowing in Linear Haskell with purity. Notably, it features parallel state mutation with affine mutable references inside pure computation, unlike the IO and ST monads and existing Linear Haskell APIs. It also enjoys purity, lazy evaluation, first-class polymorphism and leak freedom, unlike Rust. We implement Pure Borrow simply as a library in Linear Haskell and demonstrate its power with a case study in parallel computing. We formalize the core of Pure Borrow and build a metatheory that works toward establishing safety, leak freedom and confluence, with a new, history-based model of borrowing.

CCS Concepts: • **Theory of computation** → **Type theory**; • **Software and its engineering** → *Functional languages*.

Additional Key Words and Phrases: Linear Haskell, Rust-style borrowing, purity, safety, parallelism

1 Introduction

Software is the art of embodying human ideals on finite physical machines. This tension naturally leads to two major programming paradigms. One is *functional programming*, more on the side of humans. It offers high-level abstractions and safety for smooth human reasoning. The other is *imperative programming*, more geared toward machines. It offers low-level stateful operations, enabling great performance. Unifying these two programming paradigms has been a long-standing, central topic in programming language research.

Functional programming favors *purity*, or the property that the result of the computation is always the same regardless of the timing and context. Purity allows one to safely forget the physical machine state and reason about behaviors just as in mathematics. It enables safe lazy evaluation, safe concurrency, and deep program optimizations such as fusion. Moreover, a number of languages—e.g., Haskell [Hudak et al. 1992], F* [Swamy et al. 2016], and Lean [de Moura et al. 2015]—rigorously ensure that *every computation is pure*, unless marked otherwise. Especially in the presence of concurrency, purity is really helpful because it eliminates tricky, subtle concurrency bugs that occur only with low probability depending on thread scheduling.

Purity is thus desired, but naïvely it causes significant runtime overhead to keep all data persistently. This calls for unifying pure functional programming with performative, stateful imperative programming. A long-known baseline approach to this is the ST monad [Launchbury and Peyton Jones 1994]. The monad ST^s allows efficient stateful computation, but its result can still be taken out into pure computation, unlike the IO monad [Peyton Jones and Wadler 1993].

* This is an extended version of the PLDI 2026 paper [Matsushita and Ishii 2026b] with an appendix on the formal calculus and metatheory.



Table 1. High-level comparison of our Pure Borrow with existing approaches. Yes ● / No ○. *Mutate*: Supports destructive mutation. *Pure*: The result can be safely taken out in pure computation. *Parallel*: Supports safe parallel execution. *Aff. mut.*: Supports affine mutable references (● for droppable but copyable mutable references). *Leak-free*: Guarantees the absence of resource leaks.

	<i>Mutate</i>	<i>Pure</i>	<i>Parallel</i>	<i>Aff. mut.</i>	<i>Leak-free</i>
Pure computation only	○	●	●	-	-
IO monad [Peyton Jones and Wadler 1993]	●	○	●	●	○
ST monad [Launchbury and Peyton Jones 1994]	●	●	○	●	○
Rust [Matsakis and Klock 2014]	●	○	●	●	○
Pure Linear Haskell [Bernardy et al. 2018]	●	●	●	○	●
RIO monad [Bernardy et al. 2018]	●	○	●	●	●
Our Pure Borrow: BO monad	●	●	●	●	●

Another promising approach to this goal is to localize mutation using *linear* or *affine types* [Wadler 1990; Turner et al. 1995; Wansbrough and Peyton Jones 1999], requiring resources to be used exactly once (linear) or up to once (affine), inspired by Girard [1987]’s linear logic. Linear or affine types can localize the effect of state mutation, especially for safety. Also, linearity can prevent resource leaks. Haskell, a purely functional language, was recently extended with linear types by Bernardy et al. [2018], in the name of Linear Haskell. It achieves safe state mutation inside pure computation, while supporting concurrency and ensuring leak freedom unlike the ST monad. It also introduced a leak-free variant of the IO monad, called the RIO monad.

However, it remained unknown whether such a pure language as Linear Haskell could safely support *borrowing* in the sophisticated style of Rust [Matsakis and Klock 2014], a highly successful imperative language with affine types. While Rust-style borrowing originates from the long line of work on region types [Tofte and Talpin 1994; Fähndrich and DeLine 2002; Grossman et al. 2002; Fluet et al. 2006; Haller and Odersky 2010], it features a distinct form of *non-locality*: each borrower can be freely split and dropped (i.e., affine), and its ownership is non-locally returned to the lender without direct communication. This flexibility offers an advantage over the existing Linear Haskell APIs, which require direct threading of ownership.

We answer the question above affirmatively with this work, *Pure Borrow*. Our contributions are summarized as follows:

- We develop *Pure Borrow*, a novel framework that realizes Rust-style borrowing in Linear Haskell with purity. (§3). Table 1 shows a high-level comparison with existing approaches. Notably, Pure Borrow features parallel state mutation with affine mutable references inside pure computation, unlike the IO and ST monads and existing Linear Haskell APIs. It also enjoys purity, lazy evaluation, first-class polymorphism and leak freedom, unlike Rust.
- We implement Pure Borrow simply as a library in Linear Haskell and demonstrate its power by a case study on parallel quicksort (§4). Notably, our implementation compiles with the current Haskell compiler GHC 9.10+, without any modification to the compiler.
- We formalize the core of Pure Borrow and build a metatheory that works toward establishing safety, leak freedom and confluence, with a new, history-based model of borrowing (§5).

We also explain the background (§2), compare Pure Borrow with existing approaches (§6), and discuss related and future work (§7). Our Haskell implementation of Pure Borrow and our benchmark suite are archived at [Matsushita and Ishii 2026a], and the latest version is maintained at <https://github.com/SoftwareFoundationGroupAtKyotoU/pure-borrow>.

2 Background

Before diving deeply into our work, Pure Borrow, we briefly review prior approaches to local state mutation inside pure computation in Haskell, namely the ST monad and Linear Haskell.

2.1 ST Monad

The classical approach to achieve local mutation in Haskell is the ST monad [Launchbury and Peyton Jones 1994], as mentioned in §1.

```

data STs a
instance Monad STs
runST :: (∀s. STs a) → a

data VectorSTs a
newVectorST :: [a] → STs (VectorSTs a)
elemsST    :: VectorSTs a → STs [a]
writeAtST :: Int → a → VectorSTs a → STs ()
readAtST  :: Int → VectorSTs a → STs a

example :: (Int, [Int])
example = runST do
  vec ← newVectorST [0, 1, 2]
  modifyAtST 0 (+ 3) vec; modifyAtST 2 (+ 5) vec
  modifyAtST 0 (× 4) vec; n ← readAtST 0 vec
  pure (n, elemsST vec)

modifyAtST i f vec = do
  a ← readAtST i vec; writeAtST i (f a) vec

```

Fig. 1. ST monad APIs and an example usage.

Figure 1 summarizes APIs and an example usage of the ST monad. The monad $ST^s a$ is parameterized with a *region* parameter s . In the ST monad, one can freely use mutable state such as a vector $Vector_{ST}^s a$ ¹, bound to some region s . In other words, a region s represents memory locations that are allocated and kept during ST^s . Most notable is the function `runST`, which can strip off the ST monad into a pure computation. Its key trick is to use *rank-2 polymorphism* over the region variable s (technically working as a *skolem*). This aptly encapsulates a fresh region s for mutable state inside ST^s and prohibits it from leaking outside.

Although the ST monad is widely used in Haskell, it has a major limitation: it does not support parallelism. The ST monad allows shared mutable state like $Vector_{ST}^s a$. Parallelism would allow multiple threads to mutate such state. The result of computation could nondeterministically depend on thread scheduling, making `runST`, the embedding to pure computation, unsound.

2.2 Linear Haskell Before Pure Borrow

A recent groundbreaking take on purity and mutation is *Linear Haskell* [Bernardy et al. 2018], which retrofits a linear type system [Wadler 1990] onto Haskell. Notably, it is implemented as a practical, stable extension called `LinearTypes` in the Glasgow Haskell Compiler (GHC).

Linear Haskell simply adds the *linear* arrow type $a \multimap b$, along with the original unrestricted arrow type $a \rightarrow b$. Roughly speaking, the linear arrow $a \multimap b$ enforces that the argument a is used exactly once in b .² This linearity is the key to safely introducing mutable state.

Figure 2 shows core APIs and an example usage of Linear Haskell. We feature here the *linear* mutable vector type `Vector a`. Most notable is the `modifyAt_` operator. A call `modifyAt_ i f vec` performs an *in-place mutation* of the vector `vec`, applying `f` to the i -th element, and outputs the updated vector. Notably, unlike the traditional purely functional style, the old value of the vector can be safely lost. This is because there is no other alias to the original `vec`, thanks to the *linearity*

¹ The term ‘vector’ is commonly used in Haskell for a slice to an array. It amounts to the slice type `[T]` in Rust.

² Due to lazy evaluation, this is actually more subtle. Bernardy et al. [2018] describe the condition more precisely as follows: whenever the output b is consumed exactly once, the input a is consumed exactly once.

```

data Ur a where Ur :: a → Ur a
class Linearly; linearly :: (Linearly ⇒ Ur a) → Ur a
class Movable a where move :: a → Ur a

data Vector a
newVector :: Linearly ⇒ [a] → Vector a
freeVector :: Vector a → [a]
modifyAt_ :: Int → (a → a) → Vector a → Vector a
readAt_ :: Movable a ⇒ Int → Vector a → (Ur a, Vector a)

class Consumable a where consume :: a → ()
class LinearOnly a; withLinearly :: LinearOnly a ⇒ a → (Linearly ⇒ a → r) → r
par :: a → b → (a, b)

```

```

example :: Ur (Int, [Int])
example = linearly do°
  vec ← newVector [0, 1, 2]
  vec ← modifyAt_ 0 (+ 3) vec
  vec ← modifyAt_ 2 (+ 5) vec
  vec ← modifyAt_ 0 (× 4) vec
  (Ur n, vec) ← readAt_ 0 vec
  move (n, freeVector vec)

```

Fig. 2. Core Linear Haskell APIs and an example usage before Pure Borrow.

of `Vector a`. Technically, this is guaranteed by the *linear* constraint `Linearly ⇒` in the type of the `newVector` operator creating a new vector, as we explain later.

Pure do-notation. Throughout the paper, we use `do`[°]-notation for pure computations, analogous to the usual `do`-notation for monads. Roughly speaking, the monadic binding `... ← ...` in `do`[°] works like the `let`-binding `let ... = ... in`, but the former is often a better fit in Linear Haskell, because it supports *shadowing* and adopts *strict* pattern matching, unlike the latter.³ Practically, we can use this kind of custom `do`-notation by the `QualifiedDo` extension.

Opting out linearity. We introduce a special data type `Ur a` ('Ur' stands for 'Unrestricted'). It means that its content `a` can be used any number of times (i.e., be freely shared and dropped) even under linear binding, analogous to the `!`-modality in linear logic. The type class `Movable a` abstracts data types that can be moved to an unrestricted context, i.e., put under `Ur`. Some primitive types such as `()`, `Bool` and `Int` are `Movable`. Also, containers such as the list `[a]` and the pair `(a, b)` are `Movable` if the component types `a`, `b` are `Movable`. The type class `Consumable a` abstracts data types that can be consumed (but not necessarily be shared) under linear binding.

Linearly. In this paper, we use a special type class called `Linearly`, proposed by Spiwack et al. [2022]. It is introduced as a *linear constraint* `Linearly ⇒`, which is a linear version of a usual type class constraint such as `Ord a ⇒`. Essentially, `⇒` is just a variant of `→` where the argument is managed implicitly (just like `⇒` is a variant of `→` with an implicit argument).

The type class `Linearly` serves as a *witness of linearity*, or a guarantee that the current computation is inside a *linear* thunk introduced by a special operator `linearly`. Functions that create new mutable state, such as `newVector` for `Vector a`, typically take `Linearly`, because the output mutable state captures `Linearly` and thus is used `linearly`. At a high level, the operator `linearly` is analogous to the `ST` monad's `runST`: it embeds computation with mutable state into a pure context. It also introduces `Linearly`, somewhat like how `runST` introduces `s`.

To see how `Linearly` serves for safety, first imagine a fake API `newVector :: [a] → Vector a` without `Linearly` constraint. We can weaken the linear arrow of `newVector` to a usual arrow to

³ In Linear Haskell, shadowing is useful because it is common to thread through the same resource `linearly`, and strict pattern matching is often required to satisfy linearity. The `let`-binding in Haskell disallows shadowing to allow recursion such as `let ones = 1 :: ones`, just like `let rec` in ML, and adopts lazy pattern matching.

get `newVector :: [a] → Vector a`.⁴ Combined with `freeVector :: Vector a → [a]`, it easily causes a dangerous *double free*, violating memory safety. The linearity witness `Linearly` solves this. The only way to (newly) introduce `Linearly` is by calling `linearly :: (Linearly ⇒ Ur a) ⇒ Ur a`. It requests a *linear* function `Linearly ⇒ Ur a` that `linearly` takes `Linearly`. By letting `newVector` take `Linearly ⇒`, the resulting `Vector a` captures a linear resource `Linearly` and thus will not be shared unrestrictedly. See [Spiwack et al. 2022] for more details.

For utility, we also have the `LinearOnly` type class, which abstracts over data types that can be introduced only inside linear contexts, such as `Vector a`. In the presence of such data types, we can safely get `Linearly`, using the `withLinearly` operator.

The type class `Linearly` is treated very specially as a linear constraint `⇒`: it can be freely finitely duplicated or discarded (but not unrestrictedly shared), automatically by the type system.⁵

Parallelism. Pure Linear Haskell supports parallel execution, like pure non-linear Haskell. For example, `par a b` evaluates `a` and `b` in parallel and combines their outputs. The point is that the linear arrow `⇒` of `par` ensures that the mutable resources owned by `a` and `b` are disjoint. This is unlike the `ST` monad, which does not support parallelism lacking such knowledge of disjointness.

On the example. The example in Fig. 2 corresponds to that of Fig. 1. As expected, it type-checks and evaluates to `Ur (12, [12, 1, 7])`. Still, threading of `vec` is quite in the way. Although the order of updates to the 0th element matters and they cannot be swapped, updates to the 0th element and the 2nd element can be swapped and should be able to be *parallelized*. Reads can also be swapped. Threading of the whole vector `vec` interrupts such natural swapping. This motivates our work, Pure Borrow, and we come back to this point later in §3.1.

3 Overview of Pure Borrow

We now provide an overview of our Pure Borrow framework. We first give a taste of it (§3.1), showing how it can parallelize the previous example Fig. 2, and then present its details (§3.2).

3.1 Taste of Pure Borrow

Rust-style borrowing. Pure Borrow features Rust-style *borrowing* [Fähndrich and DeLine 2002; Grossman et al. 2002; Fluet et al. 2006; Haller and Odersky 2010; Matsakis and Klock 2014] safely and purely achieved in Linear Haskell. But what makes Rust-style borrowing so special?

Figure 3 illustrates the concept of Rust-style borrowing. An original owner `T` can be *mutably borrowed* to create a *mutable borrower* `Mutα T` and a *lender* `Lendα T` for a time period called the *lifetime* `α`.⁶ A mutable borrower can freely mutate the data during the lifetime `α`. Notably, borrowers are *affine*: they can be freely split (e.g., `Mutα T` into borrowers `Mutα U1`, `Mutα U2` of subregions) and dropped at any time before the end of `α`. After the lifetime `α` ends, the lender can reclaim full ownership of `T`. One might wonder how this happens *non-locally*, without direct communication of ownership from borrowers to the lender. Intuitively, ownership thrown away by borrowers is collected behind the scenes and finally returned to the lender once the lifetime `α` ends. This non-local handling of ownership and data is the distinct feature of Rust-style borrowing.

⁴ Recall that the linear arrow `⇒` is a promise by the *callee* to use the argument once, not a request to the *caller*.

⁵ As of now, linear constraints and `Linearly` have not yet landed in GHC. Still, we can already get the same functionality by introducing `Linearly` as a linear *type* (using `Linearly ⇒` instead of `Linearly ⇒`), manually handling instances of `Linearly`. Our Haskell implementation of Pure Borrow does this, and thus it compiles with GHC 9.10+. Our metatheory (§5) also follows this style for simplicity. In this paper, we use linear constraints and `Linearly` in (informal) Haskell code just as a useful automation just for the brevity of presentation.

⁶ In Rust, a mutable borrower (our `Mutα T`) is typed `&'a mut T`, but lending is only implicitly managed, lenders not being first-class citizens. Please see §6 for more detailed discussion.

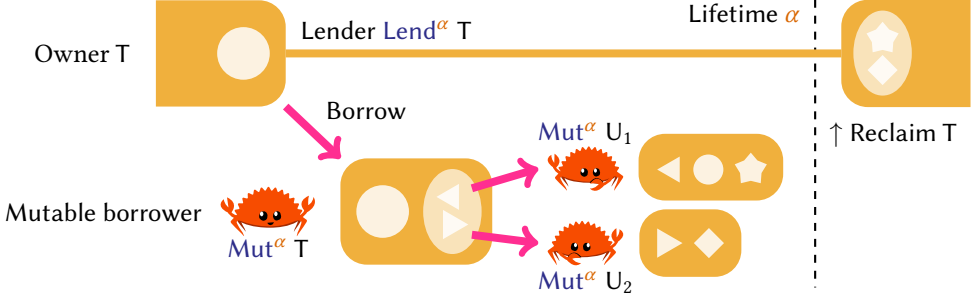


Fig. 3. Rust-style borrowing illustrated. Each Ferris corresponds to a borrower. Ferris illustrations by Karen Rustad Tölva, CC0, via <https://rustacean.net>.

Core of Pure Borrow. Now let's see how Rust-style borrowing works in Pure Borrow.

```

type BOα a; type Endα
type Mutα a; type Shareα a; type Lendα a
runBO :: Linearly ⇒
  (∀ α. BOα (Endα → a)) → a
borrow :: Linearly ⇒ a → (Mutα a, Lendα a)
share :: Mutα a → Ur (Shareα a)
reclaim :: Lendα a → Endα → a
modifyAt :: Int → (a → a) →
  Mutα (Vector a) → BOα (Mutα (Vector a))
copyAt :: Copyable a ⇒ Int →
  Shareα (Vector a) → BOα (Ur a)

```

```

1 example :: Ur (Int, [Int])
2 example = linearly doo
3   vec ← newVector [0, 1, 2]
4   runBO do
5     let !(mvec, lend) = borrow vec
6     mvec ← modifyAt 0 (+ 3) mvec
7     mvec ← modifyAt 2 (+ 5) mvec
8     mvec ← modifyAt 0 (× 4) mvec
9     let !(Ur svec) = share mvec
10    Ur n ← copyAt 0 svec
11    pure λlend → move
12      (n, freeVector $ reclaim lend end)

```

Fig. 4. Core APIs of Pure Borrow and an example usage.

Figure 4 shows the core APIs and an example usage, implementing the same logic as Figs. 1 and 2.⁷ At first, it proceeds similarly to Fig. 2 by allocating a linear mutable vector `vec` with `newVector`. But then we run a computation with borrowing using `runBO` (Line 4). Crucially, `runBO` can run `BOα` inside a *pure* computation, with a guard by rank-2 polymorphism over the lifetime α . The *BO monad* `BOα` is a novel monad we introduce in Pure Borrow, and it denotes a computation that can be run during the lifetime α . The mechanism of `runBO` and `BOα` is akin to `runST` and the `STs` monad (recall Fig. 1, §2.1). Now we `borrow` the vector `vec` under α , obtaining a mutable borrower `mvec :: Mutα (Vector Int)` and a lender `lend :: Lendα (Vector Int)` (Line 5). We can do mutations over the vector using `mvec` (Lines 6 to 8). Then we `share` the affine mutable borrower `mvec` to get a shared borrower `svec :: Shareα (Vector Int)` (Line 9) and read the value at the index 0 with `copyAt` (Line 10). Notably, `svec` is wrapped in `Ur`, so we can use it arbitrarily many times, especially for reading with `copyAt`. We are reading only once in this example, but we could add more reads without any rebinding of `svec`. Finally, we can `reclaim` the lender (Line 12) to retrieve `Vector Int`, once given by `runBO` a *dead lifetime token* `end :: Endα`, asserting that the lifetime α has ended.

⁷ The bang `!` makes a pattern strict, not lazy, and here serves to pass type checking. The reader can safely ignore it.

Parallelism. With Pure Borrow, we can go further: mutations on different elements can be performed *in parallel*. Such safe parallel mutation in pure computation is a central benefit of Pure Borrow, practically demonstrated later in §4.

```

parBO :: BOα a → BOα b → BOα (a, b)
splitAt :: Int → Borrowαk (Vector a) →
  (Borrowαk (Vector a), Borrowαk (Vector a))

let !(mvec1, mvec2) = splitAt 1 mvec
(mvec, ()) ← parBO
  (do mvec1 ← modifyAt 0 (+ 3) mvec1
   modifyAt 0 (× 4) mvec1)
  (consume <$> modifyAt 1 (+ 5) mvec2)

```

Fig. 5. Pure Borrow APIs for parallelism and an example usage, modifying Lines 6 to 8 of Fig. 4.

Figure 5 shows a parallel variant of the mutation part (Lines 6 to 8) of Fig. 4. First, we split with `splitAt` the mutable borrower `mvec` at the index 1 into two disjoint mutable borrowers `mvec1` and `mvec2`. Then we use our key API, `parBO`, for *parallel* composition of BO monads. Here, the first thread mutates the 0th element twice through `mvec1` and the second thread mutates the 2nd element (of `mvec`) once through `mvec2`. Linearity $\rightarrow$ plays a vital role in the safety of `parBO`: thanks to linearity, the ownership held by the two BO monads is mutually *disjoint*, which ensures that their mutations can safely be performed *in parallel*, while retaining *purity*.

Reborrowing. There was one thing we glossed over in the previous example Fig. 5. The new `mvec` obtained from `parBO` is actually `mvec1` in disguise. Can we do better?

Reborrowing solves this kind of situation. It creates a borrower that borrows ownership from an existing mutable borrower for a shorter lifetime.

```

reborrowing ::
  Mutα a → (∀ β. Mutβ ∧ α a → BOβ ∧ α' r) →
  BOα' (r, Mutα a)

((), mvec) ← reborrowing mvec λ mvec →
  let !(mvec1, mvec2) = splitAt 1 mvec in
  consume <$> parBO
    (do mvec1 ← modifyAt 0 (+ 3) mvec1
     modifyAt 0 (× 4) mvec1)
    (modifyAt 1 (+ 5) mvec2)

```

Fig. 6. Reborrowing API and an example usage, modifying Lines 6 to 8 of Fig. 4.

Figure 6 shows an API, `reborrowing`, and an example usage of it. The function `reborrowing` creates a mutable reference `mvec :: Mutβ ∧ α (Vector a)` that only lives during the execution of the function. Using this new `mvec`, we can perform parallel mutation just like in Fig. 5. Finally, the function restores the original mutable reference `mvec :: Mutα`, which has access to the *whole* vector. To obtain a shorter lifetime than α , we use the intersection $\beta \wedge \alpha$ of α with a fresh lifetime β .

3.2 Details of Pure Borrow

Now that we know the taste of Pure Borrow, let's take a closer look at it. Figures 7 to 14 present the central APIs of Pure Borrow. One beautiful aspect is that high-level APIs can be derived from simpler primitives, thanks to Haskell's expressivity.

Lifetimes. Figure 7 summarizes APIs for lifetimes. A lifetime $\alpha :: \text{Lifetime}$ is an atomic lifetime $\text{Al } \iota$ for some id ι , the static lifetime `Static` that lives forever ('static in Rust), or an intersection $\wedge$ of lifetimes. As we saw above, lifetime intersection plays a key role in reborrowing. Lifetime inclusion $\alpha \leq \beta$ is the partial order naturally induced, regarding $\wedge$ as the greatest lower bound and

```

data Lifetime = Al  $\iota$  | ( $\alpha :: \text{Lifetime}$ )  $\wedge$  ( $\beta :: \text{Lifetime}$ ) | Static
class ( $\alpha :: \text{Lifetime}$ )  $\leq$  ( $\beta :: \text{Lifetime}$ )

data Now $^{\alpha :: \text{Lifetime}}$ ; data End $^{\alpha :: \text{Lifetime}}$ ; instance LinearOnly Now $^{\alpha}$ ; instance Movable End $^{\alpha}$ 
newLifetime :: Linearly  $\Rightarrow$  ( $\forall \iota, \text{Now}^{\text{Al } \iota} \rightarrow a$ )  $\rightarrow a$ ; endLifetime :: Now $^{\text{Al } \iota} \rightarrow \text{Ur End}^{\text{Al } \iota}$ 

```

Fig. 7. Core APIs for lifetimes.

```

newtype BO $^{\alpha :: \text{Lifetime}}$  a; pure :: a  $\rightarrow$  BO $^{\alpha}$  a
( $\gg$ ) :: BO $^{\alpha}$  a  $\rightarrow$  (a  $\rightarrow$  BO $^{\alpha}$  b)  $\rightarrow$  BO $^{\alpha}$  b; parBO :: BO $^{\alpha}$  a  $\rightarrow$  BO $^{\alpha}$  b  $\rightarrow$  BO $^{\alpha}$  (a, b)

execBO :: Now $^{\alpha}$   $\rightarrow$  BO $^{\alpha}$  a  $\rightarrow$  (Now $^{\alpha}$ , a)    sexecBO :: Now $^{\beta}$   $\rightarrow$  BO $^{\beta \wedge \alpha}$  a  $\rightarrow$  BO $^{\alpha}$  (Now $^{\beta}$ , a)
runBO :: Linearly  $\Rightarrow$                           srunBO :: Linearly  $\Rightarrow$ 
  ( $\forall \alpha. \text{BO}^{\alpha} (\text{End}^{\alpha} \rightarrow a) \rightarrow a$ )              ( $\forall \beta. \text{BO}^{\beta \wedge \alpha} (\text{End}^{\beta} \rightarrow a) \rightarrow \text{BO}^{\alpha} a$ )
runBO bo = newLifetime  $\lambda \text{now} \rightarrow$  do $^{\circ}$           srunBO bo = newLifetime  $\lambda \text{now} \rightarrow$  do
  (now, f)  $\leftarrow$  execBO now bo                    (now, f)  $\leftarrow$  sexecBO now bo
  Ur end  $\leftarrow$  endLifetime now; f end              Ur end  $\leftarrow$  pure (endLifetime now); pure (f end)

```

Fig. 8. Core APIs for the BO monad.

Static as the maximum element. We introduce two basic tokens for lifetimes, the *live lifetime token* Now $^{\alpha}$ and the *dead lifetime token* End $^{\alpha}$.⁸ The former exclusively asserts that α is ongoing, and the latter persistently asserts that α has ended. With **newLifetime**, we can get Now $^{\text{Al } \iota}$ for a fresh id ι . Then with **endLifetime**, we can consume that to get End $^{\text{Al } \iota}$. The idea of these tokens comes from RustBelt’s lifetime logic [Jung et al. 2018a], as we discuss later.

BO monad. Figure 8 shows the APIs for our BO monad. At runtime, BO $^{\alpha}$ a is just a thunk for a, just like ST^s a, but the APIs for it are quite new. The key primitive is **execBO**, which takes a pure value out of BO $^{\alpha}$ using the live lifetime token Now $^{\alpha}$. Actually, a high-level combinator **runBO** we used above (Fig. 4) is derived from **execBO** together with **newLifetime** and **endLifetime**, using rank-2 polymorphism. Also, we have a primitive **sexecBO** and a derived combinator **srunBO** for *scoped* execution of the BO monad for sub-lifetimes introduced through intersection $\wedge$. It is used to derive the **reborrowing** combinator we used above (Fig. 6), as shown soon (Fig. 14).

Borrowing. Figure 9 lists the APIs for borrowing. We feature Mut $^{\alpha}$ a for a mutable borrower, Share $^{\alpha}$ a for a shared borrower, and Lend $^{\alpha}$ a for a lender. Notably, they are just **newtype** wrappers over the body type a, i.e., they have the same runtime representation as a.⁹ Also, any Haskell type a can be a target of borrowing. For utility, Mut $^{\alpha}$ and Share $^{\alpha}$ are unified into a general type Borrow $^{\alpha}_k$. The following are the core primitives: we can **borrow** an object to create a mutable borrower and a lender, then we can **share** a mutable borrower to get an unrestricted shared borrower, and finally we can **reclaim** the ownership from a lender once the lifetime has ended.

Vectors. Figure 10 shows the functions for accessing vectors through borrowers. The primitive **getAt** i vec takes a borrower to the i-th element of vec. The primitive **swapAt** i j vec swaps the i-th and j-th elements of vec. The primitive **updateAt** i k vec updates the i-th element of vec using k; it is quite expressive, allowing access to the BO monad inside k.

⁸ To enhance automation, our Haskell implementation actually introduces a type class for End $^{\alpha}$.

⁹ We exploit this fact to implement various borrowing operations in the subsequent APIs (e.g., **borrow**, **share** and **reclaim**) by internally using unsafe zero-cost type coercion (i.e., a representationally identity function).


```

newtype Borrow $\alpha$  :: Lifetime $\alpha$  k :: BorrowKind a
data BorrowKind = Mut | Share
type Mut $\alpha$  = Borrow $\alpha$ Mut
type Share $\alpha$  = Borrow $\alpha$ Share
newtype Lend $\alpha$  :: Lifetime $\alpha$  a

instance LinearOnly (Mut $\alpha$  a)
instance Consumable (Mut $\alpha$  a)
instance Movable (Share $\alpha$  a)

borrow :: Linearly  $\Rightarrow$  a  $\multimap$  (Mut $\alpha$  a, Lend $\alpha$  a)
reclaim :: Lend $\alpha$  a  $\multimap$  End $\alpha$   $\rightarrow$  a
share   :: Mut $\alpha$  a  $\multimap$  Ur (Share $\alpha$  a)

```

Fig. 9. Core APIs for borrowing.

```

getAt ::  $\beta \leq \alpha \Rightarrow$  Int  $\rightarrow$ 
  Borrow $\alpha$ k (Vector a)  $\multimap$  BO $\beta$  (Borrow $\alpha$ k a)
swapAt ::  $\beta \leq \alpha \Rightarrow$  Int  $\rightarrow$  Int  $\rightarrow$ 
  Mut $\alpha$  (Vector a)  $\multimap$  BO $\beta$  (Mut $\alpha$  (Vector a))
updateAt ::  $\beta \leq \alpha \Rightarrow$  Int  $\rightarrow$ 
  (a  $\multimap$  BO $\beta$  (b, a))  $\multimap$  Mut $\alpha$  (Vector a)  $\multimap$ 
  BO $\beta$  (b, Mut $\alpha$  (Vector a))

modifyAt ::  $\beta \leq \alpha \Rightarrow$  Int  $\rightarrow$  (a  $\multimap$  a)  $\multimap$ 
  Mut $\alpha$  (Vector a)  $\multimap$  BO $\beta$  (Mut $\alpha$  (Vector a))
modifyAt i f mvec = do
  ((), mvec)  $\leftarrow$  updateAt i
  ( $\lambda$  a  $\rightarrow$  pure ((), f a)) mvec
  pure mvec

```

Fig. 10. Core APIs for accessing vectors through borrowers.

```

class Copyable a; copy :: Copyable a  $\Rightarrow$  Share $\alpha$  a  $\rightarrow$  a
instance Copyable Int; instance Copyable End $\alpha$ ; instance Copyable (Share $\alpha$  a)

copyAt :: (Copyable a,  $\beta \leq \alpha$ )  $\Rightarrow$  Int  $\rightarrow$  Share $\alpha$  (Vector a)  $\rightarrow$  BO $\beta$  (Ur a)
copyAt i svec = do Ur s  $\leftarrow$  move <$> getAt i svec; pure $ Ur (copy s)

```

Fig. 11. Core APIs for copying.

```

splitPair :: Borrow $\alpha$ k (a, b)  $\multimap$  (Borrow $\alpha$ k a, Borrow $\alpha$ k b)
splitEither :: Borrow $\alpha$ k (Either a b)  $\multimap$  Either (Borrow $\alpha$ k a) (Borrow $\alpha$ k b)
splitAt :: Int  $\rightarrow$  Borrow $\alpha$ k (Vector a)  $\multimap$  (Borrow $\alpha$ k (Vector a), Borrow $\alpha$ k (Vector a))

```

Fig. 12. APIs for splitting.

Copying. We provide a mechanism called copying, as shown in Fig. 11. We can `copy` the body `a` out of a shared reference `Share $\alpha$  a` if `Copyable a` holds. It holds roughly when `a` is a persistent data type, such as `Int`, under a condition similar to but different from `Movable a`; `Copyable` does not take ownership, while `Movable` does. More specifically, any persistent data types (i.e., those without any mutable states or linear resources) can be `Copyable`.¹⁰

Splitting. Figure 12 shows primitives for splitting borrowers. Our Haskell implementation includes a generic mechanism for deriving a `split*` primitive for various data types, not only the ones shown here. We also have a splitting primitive for a vector. Notably, these splits can be performed even outside the BO monad, because the information required for them is all persistent.¹¹

¹⁰ The class `Copyable` roughly corresponds to Rust’s `Copy`, but further allows immutable boxing (like Rust’s `Box<T>` but immutable) unlike it. Notably, this enables recursive data types to be `Copyable` (e.g., `[a]` is `Copyable` under `Copyable a`).

¹¹ Note that the length of `Vector a` is immutable, as the type represents a fixed-size slice of an array.

Fig. 13. Core APIs for subtyping.

Fig. 14. Primitive and derived operators for reborrowing.

RustBelt and Pure Borrow. Jung et al. [2018a] developed RustBelt, a semantic soundness proof of Rust’s ownership type system, by building the *lifetime logic*, a library for modeling Rust-style borrowing in the *separation logic* Iris [Jung et al. 2015]. Some of Pure Borrow’s design ideas originate from RustBelt’s lifetime logic [Jung et al. 2018a]. These include lifetime tokens Now^α and End^α and the use of joinMut for reborrowing. Nevertheless, designing Pure Borrow safely required careful thought because the lifetime logic (or Iris in general) does not guarantee purity or leak freedom.

Expressivity compared to Rust. Notably, Pure Borrow supports non-lexical lifetimes [The Rust Community 2017] to some extent. Although our high-level combinators `runBO` and `srunBO` offer lexically scoped lifetimes, we also provide more primitive APIs, `execBO`, `sexecBO`, `newLifetime` and `endLifetime`, for managing lifetimes manually with lifetime tokens `Nowα` and `Endα`. This is similar to what RustBelt [Jung et al. 2018a] does to model Rust’s lifetime-based type system using lifetime tokens. Still, Pure Borrow lacks a highly automated borrow checker like Rust has, and such smooth automation in Pure Borrow is left for future work.

Some advanced borrowing patterns available in Rust are not supported in Pure Borrow, on the other hand. For example, let us consider the following Rust code:

```
enum Sum<A, B> { Left(A), Right(B) }
fn test<A, B>(mut s: Sum<A, B>, newb: B) → Option<Sum<A, B>> {
  match s { Sum::Left(a) ⇒ { drop(a); None }
            Sum::Right(ref mut b) ⇒ { *b = newb; Some(s) } }
```

This is a mixture of moving and borrowing, enabled by Rust’s `ref` pattern.¹² Pure Borrow does not support this kind of advanced borrowing pattern because it requires pattern matching to be tightly coupled with borrowing.

4 Case Study and Evaluation

We have implemented the Pure Borrow APIs presented in §3 as a pure-borrow package in Haskell, which compiles with the current Haskell compiler GHC 9.10+. In this section, we demonstrate its power with a case study: *parallel quicksort* [Hoare 1961] implemented in our library.¹³

4.1 Parallel Quicksort Implementations

We have two implementations of parallel quicksort: a naïve version and a work-stealing version. In what follows, we present these implementations and discuss the expressiveness of our APIs.

Naïve parallel quicksort. Figure 15 shows our *naïve* implementation of parallel quicksort. The function `qsort` is the main part. It creates a spark (lightweight thread) for each partitioned sub-vector, as illustrated in the figure. Its budget upper-bounds the number of sub-vectors, and it falls back to sequential execution if the budget is exhausted. The function proceeds as follows. If the size of the vector is more than one, it selects a pivot value from the middle of the vector (Line 6) with `copyAtMut`—recall that it is implemented with *reborrowing*, as shown in Fig. 14. Then it calls `divide`, which we explain soon, to partition the mutable borrower `mvec` into low and high parts (Line 8). Finally, it recurses on the sub-vectors with halved budgets (Line 10). The calls are combined using the `parIf` combinator, which invokes `parBO` (Fig. 8) to evaluate both arguments in parallel if the budget remains, or otherwise executes them sequentially.

Division. Figure 16 shows the `divide` function that partitions the given vector into low and high parts according to the pivot value. This is an almost literal translation of the `partitionBy` function from `vector-algorithms` package [Doel and de Castro Lopo 2025] with our API. The function works in two modes: `partUp` and `partDown`, which scan and grow the lower and upper ends of the active interval, respectively. When an out-of-place element is found, it swaps the elements at the two indices and switches mode with `swapAt` (Line 12). When the two indices meet, it splits the vector into two parts with `splitAt` (Lines 8 and 13) and returns them.

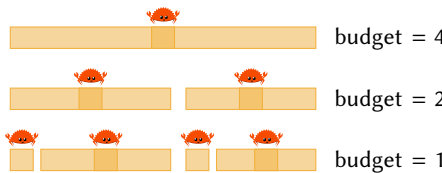
¹² In the Left case, the body `a`: `A` of `s` is moved out and dropped, whereas in the Right case, the body of `s` is mutably borrowed as `b`: `&mut B`. Therefore, `s` is partially moved out only in the first case and mutably borrowed only in the second case.

¹³ Our implementation and benchmark suite are archived at [Matsushita and Ishii 2026a], and the latest version is available at <https://github.com/SoftwareFoundationGroupAtKyotoU/pure-borrow>.

```

1 qsort :: (Ord a, Copyable a,  $\beta \leq \alpha$ )  $\Rightarrow$  Int  $\rightarrow$  Mut $^\alpha$  (Vector a)  $\rightarrow$  BO $^\beta$  ()
2 qsort budget mvec = case size mvec of
3   (Ur n, mvec) | n  $\leq$  1  $\rightarrow$  pure $ consume mvec
4   | otherwise  $\rightarrow$  do
5     let i = quot n 2
6     (Ur pivot, mvec)  $\leftarrow$  copyAtMut i mvec
7     (mlo, mhi)  $\leftarrow$  divide pivot mvec 0 n
8     let budget' = quot budget 2
9     consume <$> parlf (budget' > 0)
10    (qsort budget' mlo) (qsort budget' mhi)
11 where
12   parlf b = if b then parBO else  $\lambda$  l r  $\rightarrow$  do l  $\leftarrow$  l; r  $\leftarrow$  r; pure (l, r)

```



Each  depicts a lightweight thread spawned by parBO. Ferris: Tölva, CC0.

Fig. 15. A naïve parallel implementation of quicksort and an illustration.

```

1 divide :: (Ord a, Copyable a,  $\beta \leq \alpha$ )  $\Rightarrow$ 
2   Int  $\rightarrow$  Mut $^\alpha$  (Vector a)  $\rightarrow$  Int  $\rightarrow$  Int  $\rightarrow$  BO $^\beta$  (Mut $^\alpha$  (Vector a), Mut $^\alpha$  (Vector a))
3 divide pivot = partUp
4 where
5   partUp mvec l u
6     | l < u = do (Ur a, mvec)  $\leftarrow$  copyAtMut l mvec
7                 if a < pivot then partUp mvec (l + 1) u else partDown mvec l (u - 1)
8     | otherwise = pure $ splitAt l mvec
9   partDown mvec l u
10    | l < u = do (Ur a, mvec)  $\leftarrow$  copyAtMut u mvec
11              if pivot < a then partDown mvec l (u - 1)
12              else do mvec  $\leftarrow$  swapAt mvec l u; partUp mvec (l + 1) u
13    | otherwise = pure $ splitAt l mvec

```

Fig. 16. The implementation of divide function.

Work-stealing parallel quicksort. While the previous implementation is concise, it is rather *naïve* because `parBO` is a structural parallel composition that waits for both arguments to finish, which can lead to extra waiting time and degraded performance.

To address this issue, we develop a general, modular, safe, and efficient API for *work-stealing* parallelization of divide-and-conquer algorithms, as shown in Fig. 17a. Although the work-stealing API is designed to be used with our Pure Borrow APIs and is mostly implemented using them, it also uses some unsafe data structures, such as concurrent queues mutated by multiple threads under the hood. Hence, this API should be interpreted as a proof-of-concept demonstrating that

```

data DivideConquerα a = DivideConquer { divide :: ∀β. β ≤ α ⇒ Mutβ a → BOβ (Resultβ a) }
data Resultβ a = Done | Continue [Mutβ a]
divideAndConquer :: β ≤ α ⇒ Int → DivideConquerα a → Mutα a → BOβ (Mutα a)

```

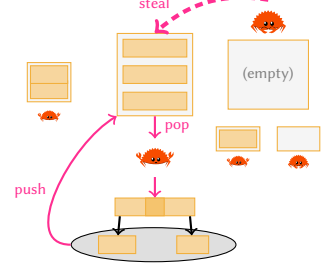
(a) General API for work-stealing parallelism.

```

qsortDC :: (Ord a, Copyable a, β ≤ α) ⇒
  Int → Int → Mutα (Vector a) → BOβ (Mutα (Vector a))
qsortDC nwork thresh = divideAndConquer nwork $
  DivideConquer λmvec → case size mvec of
    (Ur n, mvec)
    | n ≤ 1 → let () = consume mvec in pure Done
    | n ≤ thresh → Done <$> qsort 0 mvec
    | otherwise → let i = quot n 2 in do
      (Ur pivot, mvec) ← copyAtMut i mvec
      (mlo, mhi) ← divide pivot mvec 0 n
      pure $ Continue [mlo; mhi]

```

(b) Quicksort implementation.



(c) Each worker owns a deque. It pushes/pops at the top (near-side). An idle worker steals from another's bottom. Ferris: Tölva, CC0.

Fig. 17. Work-stealing parallelism.

Pure Borrow can provide good basic building blocks for more sophisticated APIs, and we do not claim that it is the best API for work-stealing parallelism. `DivideConquer` wraps a function for each step of divide-and-conquer, which takes a mutable borrower, executes in the `BO` monad, and returns `Result`, which is either `Done` or `Continues` with divided parallelized pieces of work. Given `DivideConquer`, `divideAndConquer` parallelizes the whole divide-and-conquer algorithm with a work-stealing scheduler. As illustrated in Fig. 17c, each worker owns a local double-ended queue of jobs, and whenever a thread finishes a job, its sub-jobs are pushed to the top. Then an idle worker first tries to pop another piece of work from the top of its own deque, and tries to *steal* from the bottom of another's queue when its own queue is empty. We also allocate a condition variable for each piece of work for synchronization. We carefully design `divideAndConquer` so that concurrent mutation is safely encapsulated, keeping the observed behavior pure.

Figure 17b gives a more sophisticated implementation of parallel quicksort with this work-stealing scheduler. It retains almost the same logic as the naïve implementation Fig. 15, but returns “divided” pieces of work instead of recursing on the sub-vectors. If the size of the vector is less than a given threshold, it falls back to sequential quicksort.

Notably, this is efficiently parallelized thanks to the absence of post-processing, which benefits greatly from *affine* mutable borrowers in Pure Borrow. Traditional move-based APIs would require post-processing to join vectors (see [Splitting with reversion](#) in §6 for a detailed discussion).

4.2 Benchmark Results

To evaluate performance, we measured the runtime and memory allocation for randomly generated integer vectors of varying sizes. We compared our two implementations with the `introsort`¹⁴ implementation in the `vector-algorithms` package [Doel and de Castro Lopo 2025], widely used in industry. More concretely, we compare the following implementations: **Introsort**, industrial-level sequential introsort from the `vector-algorithms` package; **Sequential**, sequential quicksort,

¹⁴ Introsort [Musser 1997] is a variant of quicksort that cleverly switches to heap or insertion sort to improve performance.

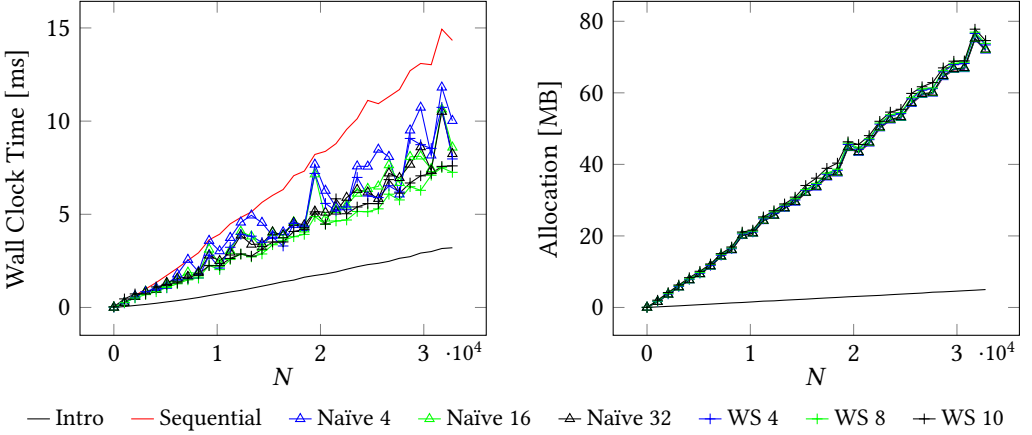


Fig. 18. The performance of parallel quicksort implementations.

i.e., our naïve quicksort in Fig. 15 with budget = 0, serving as the baseline; **Naïve** N , our naïve parallel quicksort (Fig. 15) with budget = N , dividing the whole vector into at most N pieces; and **WS** N , our work-stealing parallel quicksort (Fig. 17) with thresh = 16 and N worker threads. The benchmarks were run with the tasty-bench framework [Lelechenko 2021] on a MacBook Pro 2024 with an Apple M3 chip (4 performance cores and 6 efficiency cores) and 16 GB of RAM. All cases were run with the `+RTS -N10` option, enabling parallelism on 10 cores.

Figure 18 shows the benchmark results. Overall, our parallel quicksort implementations have considerably faster runtimes than the sequential quicksort, and our work-stealing versions largely outperform the naïve ones in runtime. These results are promising and demonstrate the eligibility of our Pure Borrow APIs as building blocks for sophisticated parallelism. Although our current parallel quicksort implementations do not outperform the industrial-level introsort, we believe that more sophisticated implementations can further improve performance.

4.3 Summary

Our Pure Borrow framework enables writing safe, pure, and efficient algorithms with parallelized mutation concisely in Linear Haskell. Our parallel quicksort implementations written in our Pure Borrow APIs showed promising performance.

5 Formal Calculus and Metatheory

We have seen above that our Pure Borrow framework is powerful, providing safe, rich abstraction for efficient mutation. But is this powerful extension to Linear Haskell really *safe*?

To answer this question, we formalize Linear Haskell extended with Pure Borrow (§5.1, §5.2) and establish a metatheory for its safety, leak freedom and confluence (§5.5). For purity, we develop two operational semantics, one for physical behavior and the other for mathematical modeling (§5.3), and formulate a type-based relation between the two (§5.4).

5.1 Program Syntax

Figure 19 shows the program syntax.¹⁵ We keep the language minimal while supporting the core functionalities of borrowing. Values of the BO monad are constructed with monad constructors *mo*.

¹⁵ The small numbers (e.g., 2 in (+) 2) show the arity. For simplicity, we don't have partial application to *op*, *mo*, and *C*.

Fig. 19. Program syntax.

Fig. 20. Syntax for the type system.

¹⁶ For each type constructor F , we assume a data type declaration, which can be recursive.

Value $v ::= \lambda x \rightarrow t \mid n \mid C \bar{x} \mid mo \bar{x}$	
Evaluation context $K ::= [\cdot] \mid [\cdot] x \mid \text{seq} [\cdot] x \mid \text{case} [\cdot] \text{ of } \{ \overline{C \bar{x} \rightarrow t} \} \mid op \bar{x} [\cdot] \bar{y}$	
(a) Common things.	
Location ℓ Memory $M ::= \overline{\ell \mapsto x}$	Borrower constr. $\dot{B} ::= \text{Mut}^{\dot{P}} \mid \text{Share}$
Extra value $\hat{d} ::= \emptyset \mid \text{Ref } \ell \mid \text{Done } x$	Extra value $\dot{d} ::= \emptyset \mid \emptyset_H \mid \text{Ref } x$ $\mid \text{Lend}^{\delta} x \mid \text{Done}_H x$
Extra operator $\hat{op} ::= \dots$	Extra operator $\dot{op} ::= \dots$
Term $\hat{t}, \hat{u} ::= t \mid \hat{d} \mid \hat{op} \bar{x}$	Term $\dot{t}, \dot{u} ::= t \mid \dot{d} \mid \dot{op} \bar{x} \mid \dot{B} \dot{t}$
Value $\hat{v}, \hat{w} ::= v \mid \hat{d}$	Value $\dot{v}, \dot{w} ::= v \mid \dot{d} \mid \dot{B} \dot{v}$
Evaluation context $\hat{K} ::= K \mid \hat{op} \bar{x} [\cdot] \bar{y}$	Evaluation ctx. $\dot{K} ::= K \mid \dot{op} \bar{x} [\cdot] \bar{y} \mid \dot{B} \dot{K}$
Env. $\hat{E} ::= \overline{x = \hat{t}}$ Config. $\hat{G} ::= \hat{E}; M; x$	Env. $\dot{E} ::= \overline{x = \dot{t}}, \bar{\delta}$ Config. $\dot{G} ::= \dot{E}; x$
(b) For mutative semantics.	(c) For denotational semantics. Also see Fig. 22.

Fig. 21. Core syntax for the two operational semantics.

operational semantics. It is free of the global memory state and instead locally manages information about each part of the state according to ownership. This localizes mutation and executes much like purely functional languages. Indeed, denotational operational semantics is designed to be *strongly confluent* (Theorem 5.1, §5.5), and thus serves as the witness for the *purity* of our Pure Borrow. Later, we establish an *association system* (§5.4), or a type-based relation between the two execution models, and state that the relation is actually a *bisimulation* (Conjecture 5.5, §5.5), which entails that well-typed programs behave the same way in the two execution models. This overall scheme largely follows the purity proof for original Linear Haskell by Bernardy et al. [2018],¹⁷ but further introduces a new mechanism for justifying Rust-style borrowing: *histories*.

Histories for borrowing. As explained in §3.1 (and illustrated in Fig. 3), Rust-style borrowing is great in that no direct communication is needed for borrowers to return ownership to the lender. However, this poses a challenge in building a *pure* execution model. How can the lender know the final result of mutation by the borrowers, when it reclaims the ownership?

To solve this, we propose a new approach: recording the *history* of updates on borrowed data.¹⁸ More concretely, our denotational operational semantics goes as follows. Because each mutation to borrowed data (more specifically, `updateRef`) happens inside the BO monad, executed with `execBO/sexecBO` threading live lifetime tokens `Nowα`, we can record the mutation history to the tokens. The history can then be inherited by dead lifetime tokens `Endα` at `endLifetime`. When the lender *reclaims* the ownership, we can restore the final version of the borrowed content from the history saved in the dead lifetime token. This successfully models global communication from the borrowers to the lender in a *local* way.

Overview. Now that we have a big picture, let's dive into the two operational semantics. Figures 21 and 22 present the syntax for them. More details are presented in Appendix A.2.

¹⁷ The ordinary and denotational dynamic semantics of Bernardy et al. [2018] correspond to our mutative and denotational operational semantics, respectively.

¹⁸ Other studies used different methods, such as prophecies, to model mutable borrowing functionally. Please refer to §7 for a comparison of our history-based approach with these existing methods.

Borrow id $\text{BorId} \ni \delta$ Borrow path $\text{BorPath} \ni p, q ::= \delta \mid p.i$
 History $\text{Hist} \ni H ::= \overline{p \leftarrow x}$

Fig. 22. Syntax for histories in denotational semantics.

$$\begin{aligned} E &\triangleq x = 3, \text{main} = \text{execBO now } (\text{modifyRef } (\lambda x \rightarrow x + 4) \text{ ref}) \\ E' &\triangleq x = 3, y = 7, \text{main} = (\text{now}', \text{ref}') \\ \text{ref} = \text{Ref } \ell, \text{now} = \emptyset, E; \ell \mapsto x; \text{main} &\quad \text{ref} = \text{Mut}^\delta(\text{Ref } x), \text{now} = \emptyset_\emptyset, E; \text{main} \\ \rightarrow^* \text{ref}' = \text{Ref } \ell, \text{now}' = \emptyset, E'; \ell \mapsto y; \text{main} &\quad \rightarrow^* \text{ref}' = \text{Mut}^\delta(\text{Ref } y), \text{now}' = \emptyset_{\delta \leftarrow y}, E'; \text{main} \end{aligned}$$

Fig. 23. Reduction example in mutative and denotational semantics.

To model Haskell’s lazy, call-by-need evaluation, both semantics are formulated in the style of Launchbury [1993], where the environment consists of variable-term bindings. Notably, the operational semantics has a nondeterministic evaluation order, as clarified by the syntax of evaluation contexts (K etc.), where the arguments of the operators (op etc.) are evaluated in *parallel*. Despite this freedom, our language satisfies the confluence and behavior uniqueness, as discussed later (Theorem 5.1 and Corollary 5.8, §5.5).

One major difference between the two semantics is that mutative semantics is equipped with the global memory M , while denotational semantics lacks it. A reference Ref is modeled as the location ℓ in the former and the content x in the latter.

To support borrowing, denotational semantics handles the *history* H , which is defined in Fig. 22. A *borrow id* δ uniquely identifies a borrow created with each *borrow*. A *borrow path* p represents a sub-component of a borrowed object. A *history* H records updates $p \leftarrow x$ of the content of a reference at p into x (by *updateRef*).¹⁹ In denotational semantics, live and dead lifetime tokens are models as $\emptyset_H$ with a history H , and each mutable borrower puts on $\text{Mut}^{\bar{p}}$, where $\bar{p}$ is initially set to the borrow id δ just after *borrow*. Notably, we can freely handle nested borrowers (i.e., borrowers that contain borrowers), thanks to the multiple paths $\bar{p}$ in the constructor $\text{Mut}^{\bar{p}}$ (each path p_j corresponds to a different layer of borrowing). Arbitrary (possibly recursive) algebraic data types within borrowers are also supported, using field accesses $.i$ in borrow paths p for splitting (Fig. 12).²⁰

Based on this syntax, we introduce the small-step reduction relations $\hat{G} \rightarrow \hat{G}'$ and $\dot{G} \rightarrow \dot{G}'$ for the two operational semantics. The reduction rules are rather straightforward, with the big picture in mind. Please refer to Appendix A.2 for the details. Figure 23 presents a reduction example in mutative and denotational semantics,²¹ showcasing how mutation works.

5.4 Association System

Mutative and denotational operational semantics are designed so that their execution is parallel and related, as informally observed in Fig. 23. We formalize this idea into what we dub the *association system*. Roughly speaking, it is a variant of the type system that is dynamic—i.e., considering intermediate execution steps—and relational—i.e., associating the two execution models.

¹⁹ A history only records the latest update for each borrow path p . So a history can have at most one item of the form $p \leftarrow x$ for each borrow path p . The order of items in a history is ignored. We write $\emptyset$ for the empty history.

²⁰ As for functions, our calculus does not offer special operations on mutable borrowers over functions. Extending Pure Borrow with a feature like Rust’s `FnMut` is left for future work.

²¹ Here, *modifyRef* is a function derived from *updateRef*, just like *modifyAt* in Fig. 10, §3.2. For readability, we omit some irrelevant variables in the environments. For both semantics, the variables *ref* and *now* are actually included in the final environment, with the same binding as the initial environment.

The high-level association judgment we provide is $\hat{G} \propto_T \dot{G}$, stating that the mutative and denotational configurations $\hat{G}, \dot{G}$ are related, under the return type T . To define this, we introduce a term-level association judgment $\dot{\Gamma} \vdash \hat{t} \propto \dot{t} :: \dot{T}$, which is defined compositionally much like the typing judgment $\Gamma \vdash t :: T$. Here, $\dot{\Gamma}$ and $\dot{T}$ are an extended version of Γ and T , where $\dot{\Gamma}$ can contain some ‘ghost resources’ that account for advanced types (e.g., $\text{MUT}^{\bar{P}}$ for $\text{Mut}^{\alpha} T$). The design of our association system is highly subtle, primarily due to lazy evaluation, first-class polymorphism, and the complex nature of Rust-style borrowing. Please refer to [Appendix B](#) for the details.

5.5 Metatheory

Finally, we introduce our metatheory for Pure Borrow, which works toward establishing memory safety, leak freedom and confluence (or behavior uniqueness). With properly designed formalization, especially denotational operational semantics and the association system, we can aim for the desired properties. Please refer to [Appendix C](#) for the omitted details.

Strong confluence of denotational operational semantics. Remarkably, by design, our denotational operational semantics is *strongly confluent*, i.e., satisfies the *diamond property*:

THEOREM 5.1 (STRONG CONFLUENCE OF DENOTATIONAL OPERATIONAL SEMANTICS). *For any denotational configurations $\dot{G}, \dot{G}_1$ and $\dot{G}_2$ such that $\dot{G}_1 \neq \dot{G}_2$, $\dot{G} \rightarrow \dot{G}_1$ and $\dot{G} \rightarrow \dot{G}_2$, there exists a configuration $\dot{G}_+$ such that $\dot{G}_1 \rightarrow \dot{G}_+$ and $\dot{G}_2 \rightarrow \dot{G}_+$.*

On the association system. Crucially, well-typedness implies association:

THEOREM 5.2 (WELL-TYPEDNESS IMPLIES ASSOCIATION). *If $\vdash t :: T$, then $(x = t; ; x) \propto_T (x = t; x)$.*

We use the term *safety* on configurations for the existence of an association. More concretely, a mutative configuration $\hat{G}$ is safe if $\hat{G} \propto_T \dot{G}$ for some $\dot{G}$ and T , and a denotational configuration $\dot{G}$ is safe if $\hat{G} \propto_T \dot{G}$ holds for some $\hat{G}$ and T .

Remarkably, safety ensures *leak freedom* for a mutative configuration:

THEOREM 5.3 (SAFETY ENSURES LEAK FREEDOM). *If $x = \hat{v}, \hat{E}; \bar{M}; x$ is safe, then $\bar{M}$ is empty.*

Progress and bisimulation. Crucially, we expect the association system to satisfy *progress* and *bisimulation*. This is a relational version of the well-known progress and preservation, the syntactic approach to type soundness [[Wright and Felleisen 1994](#)].

We carefully designed our formalization to satisfy these properties and wrote rough proof sketches for them. That said, they are currently conjectures, whose rigorous proofs are left as future work and possibly require minor fixes to the association system.

CONJECTURE 5.4 (SAFETY ENSURES PROGRESS). *For any safe configuration $\hat{G}$ that is not a normal form, there exists $\hat{G}'$ such that $\hat{G} \rightarrow \hat{G}'$. Also, for any safe configuration $\dot{G}$ that is not a normal form, there exists $\dot{G}'$ such that $\dot{G} \rightarrow \dot{G}'$.*

CONJECTURE 5.5 (ASSOCIATION IS A BISIMULATION). *For any type T , the relation $\propto_T$ is a bisimulation with respect to mutative and denotational operational semantics.*

Bisimulation clearly implies preservation of safety:

COROLLARY 5.6 (PRESERVATION OF SAFETY). *Assume [Conjecture 5.5](#). If $\hat{G}$ is safe and $\hat{G} \rightarrow \hat{G}'$ holds, then $\hat{G}'$ is safe. Analogously, if $\dot{G}$ is safe and $\dot{G} \rightarrow \dot{G}'$ holds, then $\dot{G}'$ is safe.*

Behavior uniqueness. Finally, with the bisimulation (Conjecture 5.5) and the confluence of denotational operational semantics (Theorem 5.1), we can establish the *behavior uniqueness* of mutative operational semantics under safety.²²

Definition 5.7 (Behavior in mutative operational semantics). We say $\hat{G}$ may be non-terminating if there exists an infinite reduction sequence from $\hat{G}$. We say $\hat{G}$ may terminate if there exists a reduction sequence from $\hat{G}$ to some normal form. We say $\hat{G}$ may return n if there exists a reduction sequence from $\hat{G}$ to some configuration of the form $x = n, \hat{E}; M; x$.

COROLLARY 5.8 (BEHAVIOR UNIQUENESS IN MUTATIVE OPERATIONAL SEMANTICS). Assume Conjecture 5.5. Take any safe configuration $\hat{G}$. It is not the case that $\hat{G}$ may be non-terminating and also may terminate. If $\hat{G}$ may return n and m , then $n = m$.

PROOF. By Conjecture 5.5 and Theorem 5.1. □

6 Comparison with Existing Approaches

We compare our Pure Borrow with other existing approaches to borrowing and related mechanisms for flexible resource management.

Programmable lifetime control. Rust’s borrow checker is highly automated but is largely a black box hard to intervene with. In contrast, Pure Borrow provides programmers with great control of lifetimes. A core idea is to use lifetime tokens Now^α , End^α that represent lifetime state, inherited from the semantic model of RustBelt [Jung et al. 2018a]. Thanks to Haskell’s advanced features, such as first-class polymorphism, programmers can build abstractions as observed in §3. That said, more automation in Pure Borrow would also be helpful, which is left for future work.

First-class lenders. In Rust, lenders are not first-class citizens and cannot be passed as arguments to or returned from functions. This imposes practical limitations, e.g., in handling self-borrowing data types. In contrast, our Pure Borrow treats lenders as first-class citizens, providing the first-class lender type $\text{Lend}^\alpha T$. First-class lenders are quite underexplored so far,²³ and it would be interesting to explore their practical usage. For example, Pure Borrow can represent an integer vector self-borrowing some element of it as $\exists \alpha. (\text{Mut}^\alpha \text{Int}, \text{Lend}^\alpha (\text{Vector Int}), \text{Now}^\alpha)$.

Leak freedom. In Rust, types are affine, not linear. One can accidentally cause leaks in Rust, for example by constructing cyclic references under reference-counting GC [Ben-Yehuda 2015]—a finding nicknamed as *Leakpocalypse*. A possible improvement is to keep track of leakable types with a special trait $T: \text{Leak}$,²⁴ but this has not been adopted so far for technical reasons [Matsakis 2025]. In contrast, our Pure Borrow is carefully designed to guarantee leak freedom.²⁵

²² One may wonder if the *strong confluence* of mutative operational semantics under safety derives from Conjecture 5.5 and Theorem 5.1. Unfortunately, that is not the case, because the association α_T is not left-unique. Indeed, the strong confluence does not hold in a usual sense; for example, two reductions of distinct `newRefs` may unfortunately allocate the same location ℓ . So instead, we establish the uniqueness of behavior.

²³ First-class lenders were partly supported by Nakayama et al. [2024] in the context of type-based automated verification, but they did not even support nested references.

²⁴ For example, integers `i32` and mutable references `&mut U` satisfy `Leak`. Leak freedom would then be guaranteed by imposing `T: Leak` on smart pointer types under reference-counting GC such as `Rc<T>` and `Arc<T>`.

²⁵ To be fair, Rust appears to be leak-free when restricted to the subset of features currently supported by Pure Borrow. Still, we are not aware of any existing work that formally establishes the leak freedom of Rust for such a subset, and we conjecture that Pure Borrow could be safely extended to features like reference-counting GC with leak freedom by introducing a type class for leakable types.

Linearity	comonad many ≤ once	$Ur\ a \multimap a$	comonad	$Ur\ a \multimap Ur\ (Ur\ a)$
Uniqueness	unique ≤ aliased monad	$a \multimap (\text{Mut}^\alpha\ a, \text{Lend}^\alpha\ a)$	$\text{Mut}^\alpha\ (\text{Mut}^\beta\ a) \multimap \text{Mut}^{\alpha \wedge \beta}\ a$	monad-like

Fig. 24. Linearity and uniqueness axes in OCaml and Pure Borrow.

Purity. A crucial design choice of our Pure Borrow is that it focuses on a pure fragment. As a result, unlike Rust, it does not support mutable state shared across threads such as a mutex, because the result of concurrent mutation depends on nondeterministic thread scheduling. Still, this does not preclude sequentially shared mutable state, like $\text{Vector}_{ST}^s\ a$ in the ST monad (§2.1). Indeed, we observe that Pure Borrow can support shared mutable state by requiring a non-shareable token for each mutation to such state, an approach akin to linear constraints in Haskell [Spiwack et al. 2022] and GhostCell in Rust [Yanovski et al. 2021].

Oxidized OCaml. Recently, inspired by the success of Rust, there is a movement to “oxidize” OCaml, i.e., strengthen OCaml with resource-related information for better safety and performance [Lorenzen et al. 2024; Georges et al. 2025]. This idea has been embodied as a practical language extension, OxCaml [Jane Street 2025], already in use for production code.

While this has a similar spirit to our Pure Borrow, their goal is quite different. First, they do not care much about rigorous purity, as OCaml allows side effects anywhere, unlike Haskell. Also, they aim to keep annotations for resources very lightweight, even maintaining backward compatibility with legacy OCaml. They add coarse-grained labels about resources, called *modes*. We note that their lifetime-related modes are very coarse-grained: just **local** and **global**. They provide an API for temporarily using a **global** object **locally**, which is somewhat like simple borrowing but without fine-grained scoping. Rust-style borrowing of our Pure Borrow is much more expressive and fine-grained, especially with the support of reborrowing.

An interesting high-level comparison between our approach and theirs can be made in the view of linearity and uniqueness, as summarized in Fig. 24. As clarified by Marshall et al. [2022], linearity is the inability to create (and drop) an alias, and uniqueness is the absence of aliases. OxCaml introduces modes **many** vs. **once** for linearity and **unique** vs. **aliased** for uniqueness. Lorenzen et al. [2024] point out that **many** is a comonad and **aliased** is a monad. Linear Haskell has the linearity axis, with Ur being a comonad and an exact counterpart of **many**. Notably, Rust-style borrowing of our Pure Borrow can be understood as a mechanism for allowing aliases, akin to **aliased**, but with access separated by lifetimes. Borrowers $\text{Mut}^\alpha/\text{Share}^\alpha$ and lenders Lend^α assert a possible presence of lifetime-delimited aliases—a lender for Mut^α , borrowers for Lend^α , and a lender and shared borrowers for Share^α . Roughly speaking, borrowing **borrow** is like monadic introduction, and borrower joining **joinMut** is like monadic joining. Still, Mut^α is not exactly an (indexed) monad, because it is not a functor, being invariant in subtyping (Fig. 13, §3.2). This view of borrowing as aliasing provides an exciting perspective for future work.

Splitting with reversion. Spiwack et al. [2022] proposed an approach to splitting ownership with a mechanism for reverting that. They introduce, for example, a vector type $\text{Vector}\ a\ n$ with a phantom parameter n for an abstract address. In their API, the vector can be split into two vectors $\text{Vector}\ a\ l$ and $\text{Vector}\ a\ r$ for some l and r under a peculiar constraint $\text{Slices}\ n\ l\ r$, and they can be later joined back under that constraint. The core drawback of this approach is that the structure of a split should be *statically* determined and tracked, so that phantom post-processing can be performed

later to revert the split and join tokens. In contrast, our Rust-style borrowing enables dynamic, flexible splitting of ownership, without post-processing for reversion.

Destination passing. Destination passing [Minamide 1998] is a long-known idea for achieving efficient mutation in functional languages, and its type-safe formulation has been recently explored [Bagrel 2024; Bagrel and Spiwack 2025]. At a high level, destination passing works like borrowing: the type $S \ltimes ([T_1] \otimes \cdots \otimes [T_n])$ roughly denotes an object S lending its components $T_1, \dots, T_n$ as holes. Its core difference from our Pure Borrow is that one should directly modify the lender type (on the right-hand side of $\ltimes$) each time one returns ownership to fill a hole.

Rust-style borrowing in functional languages. Wagner et al. [2025] presented how to augment a linear type system of an ML-style functional language to support Rust-style borrowing. They also proved the soundness of the resulting type system semantically using a new separation logic that supports borrowing. This work is interesting, and their high-level goal is relevant to ours. However, they did not clarify the purity of Rust-style borrowing, as their calculus is impure and their metatheory does not account for functional behavior. Also, they developed a new type system with special typing rules for borrowing and lifetimes, whereas our Pure Borrow works in existing Linear Haskell without modification.

Monadic regions. Some studies on monadic regions [Fluet and Morrisett 2004, 2006; Kiselyov and Shan 2008] introduced an extension of the ST monad with region inclusion for better resource management. While they lack leak freedom, parallelism and borrowing, their monad is, at a high level, akin to our BO monad, with region inclusion corresponding to (the dual of) lifetime inclusion.

7 Related and Future Work

Pure model of mutable borrowing. Our use of histories for purely modeling mutable borrowing is simple but novel, and interestingly different from existing known approaches.

A notable existing technique proposed by RustHorn [Matsushita et al. 2020, 2021] is to model Rust’s mutable borrowing functionally by *prophecies* [Abadi and Lamport 1988]. Prophecies are a kind of dual of histories and allow getting information about the *future*, instead of the past, in advance. In RustHorn, each mutable borrower is modeled using a prophecy that ‘foresees’ the final value of the borrowed content. Prophecies can model mutable borrowing in a compact way and thus are useful in automated functional verification, as demonstrated by RustHorn and its descendant Creusot [Denin et al. 2022]. However, prophecies are not suitable for our denotational operational semantics, because they require nondeterminism and a precise analysis of the time point at which the ownership is released.

Another existing approach is to translate Rust programs into purely functional languages. This has been explored by Electrolysis [Ullrich 2016] and Aeneas [Ho and Protzenko 2022; Ho 2024]. However, they are quite restrictive in terms of the supported patterns of mutable borrowing.²⁶ Unlike such existing approaches, our model of mutable borrowing using histories can purely model arbitrary borrowing patterns.

Type soundness proof of Rust-style borrowing. Our metatheory works toward proving type soundness using an association system, a binary version of a type system, and establishes progress and bisimulation (a variant of preservation) in the style of Wright and Felleisen [1994]. At a high level, this strategy is similar to the correctness proof of RustHorn [Matsushita et al. 2020, 2021], which

²⁶ Aeneas cannot handle nested mutable borrowers (e.g., `&mut &mut i32`) as inputs or outputs of functions [Ho 2024, Chapter 10]. Electrolysis also has this restriction [Ullrich 2016, §6.2].

proved the equivalence of Rust programs and their prophecy-based semantics through a bisimulation. Still, technically, our proof strategy is quite different from theirs. We deal with *histories* instead of prophecies, and also tackle advanced features of Haskell, such as first-class polymorphism and lazy evaluation, unlike for Rust.

RustBelt [Jung et al. 2018a] and its descendants [Dang et al. 2020; Matsushita et al. 2022; Gähler et al. 2024; Matsushita and Tsukada 2025] give a semantic, extensible soundness proof for Rust’s ownership type system extended with some APIs based on the technique of semantic typing, or logical relations [Timany et al. 2024], using the separation logic Iris [Jung et al. 2015, 2018b]. Finding such a semantic soundness proof for our Pure Borrow framework is a major challenge left for future work. All known separation logics that support borrowing, including Iris, are fundamentally affine and do not guarantee leak freedom. There exists a leak-free separation logic that supports invariants [Bizjak et al. 2019], but not yet one that supports borrowing. Also, little exploration has been done on separation logic supporting lazy evaluation.

Proving purity. Our metatheory §5.5 works toward proving the behavior uniqueness of the physical execution model, mutative operational semantics, via a bisimulation with a special execution model, denotational operational semantics, which is designed to be strongly confluent. However, it does not completely establish purity in a strict sense, because denotational operational semantics has a subtle side effect: generation of a fresh borrow id δ . The effect of id generation has been theoretically studied for ν -calculus [Pitts and Stark 1993], exploring non-trivial program equations (contextual equivalences) as well as game semantics [Abramsky et al. 2004]. As for Pure Borrow, we conjecture that purity holds despite id generation, thanks to access to each id δ being restricted to the mutable borrowers and the lender, which can live only inside the linear thunk for the borrow. A significant challenge for future work is to formulate what purity under linearity is in the first place and prove that Pure Borrow indeed satisfies it.

Equational theory. One benefit of the purity of computation is strong support for an equational theory, which establishes non-trivial contextual equivalences between programs. It is left for future work to develop a good equational theory for our Pure Borrow framework, prove its soundness, and explore optimization and verification based on the theory.

One challenge is that the denotations of programs naïvely induced from our denotational operational semantics §5.3 (or the denotational dynamic semantics of Bernardy et al. [2018]) do not satisfy full abstraction (i.e., contextually equivalent programs can have different denotations), due to linearity constraints on well-typed contexts.²⁷

Composability with other linear effects. It is interesting to explore what kind of linear effects our BO monad can compose with. This is non-trivial, especially because BO supports a *parallel* execution primitive `parBO`. We conjecture that the monad is composable with the Reader r monad for a duplicable type. Further exploration is left for future work.

Acknowledgments

We would like to express our sincere gratitude to Yudai Tanabe, Taro Sekiyama and Atsushi Igarashi for their valuable feedback and suggestions during the early stages of this work. We heartily thank Arnaud Spiwack for his insightful and encouraging feedback. We also thank the anonymous reviewers for their valuable feedback. This research was supported in part by the Hakubi Project

²⁷ For example, let us consider two functions $f :: \lambda !_ \rightarrow 0$ and $g :: \lambda !_ \rightarrow 42$ of the type $\text{Vector } () \rightarrow \text{Int}$. They would have different denotations naïvely in our denotational semantics. However, they are contextually equivalent under the linear type system, because the type system disallows sharing a linear vector $\text{Vector } ()$ under an unrestricted arrow $\rightarrow$ (rather than $\multimap$), making it impossible to call f and g to distinguish them.

at Kyoto University and JSPS KAKENHI Grant Numbers JP24KJ0133 and JP20H00582 for the first author, who is also hosted by MPI-SWS as a JSPS Overseas Research Fellow.

Data Availability Statement

Our Haskell implementation of Pure Borrow and our benchmark suite are archived on Zenodo at [Matsushita and Ishii 2026a]. The latest version is maintained as a public repository at <https://github.com/SoftwareFoundationGroupAtKyotoU/pure-borrow>.

References

- Martín Abadi and Leslie Lamport. 1988. The Existence of Refinement Mappings. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS '88)*, Edinburgh, Scotland, UK, July 5-8, 1988. IEEE Computer Society, 165–175. doi:10.1109/LICS.1988.5115
- Samson Abramsky, Dan R. Ghica, Andrzej S. Murawski, C.-H. Luke Ong, and Ian David Bede Stark. 2004. Nominal Games and Full Abstraction for the Nu-Calculus. In *19th IEEE Symposium on Logic in Computer Science (LICS 2004)*, 14-17 July 2004, Turku, Finland, Proceedings. IEEE Computer Society, 150–159. doi:10.1109/LICS.2004.1319609
- Thomas Bagrel. 2024. Destination-passing style programming: a Haskell implementation. In *35es Journées Francophones des Langages Applicatifs (JFLA 2024)*. Saint-Jacut-de-la-Mer, France. <https://inria.hal.science/hal-04406360>
- Thomas Bagrel and Arnaud Spiwack. 2025. Destination Calculus: A Linear λ -Calculus for Purely Functional Memory Writes. *Proc. ACM Program. Lang.* 9, OOPSLA1 (2025), 253–279. doi:10.1145/3720423
- Ariel Ben-Yehuda. 2015. *std::thread::JoinGuard (and scoped) are unsound because of reference cycles — Rust Issue #24292*. <https://github.com/rust-lang/rust/issues/24292>
- Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: Practical Linearity in a Higher-Order Polymorphic Language. *Proc. ACM Program. Lang.* 2, POPL (2018), 5:1–5:29. doi:10.1145/3158093
- Ales Bizjak, Daniel Gratzer, Robbert Krebbers, and Lars Birkedal. 2019. Iron: Managing Obligations in Higher-Order Concurrent Separation Logic. *Proc. ACM Program. Lang.* 3, POPL (2019), 65:1–65:30. doi:10.1145/3290378
- Hoang-Hai Dang, Jacques-Henri Jourdan, Jan-Oliver Kaiser, and Derek Dreyer. 2020. RustBelt Meets Relaxed Memory. *Proc. ACM Program. Lang.* 4, POPL (2020), 34:1–34:29. doi:10.1145/3371102
- Nils Anders Danielsson and Thorsten Altenkirch. 2010. Subtyping, Declaratively. In *Mathematics of Program Construction, 10th International Conference, MPC 2010, Québec City, Canada, June 21-23, 2010. Proceedings (Lecture Notes in Computer Science, Vol. 6120)*, Claude Bolduc, Josée Desharnais, and Béchir Ktari (Eds.). Springer, 100–118. doi:10.1007/978-3-642-13321-3_8
- Leonardo Mendonça de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean Theorem Prover (System Description). In *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9195)*, Amy P. Felty and Aart Middeldorp (Eds.). Springer, 378–388. doi:10.1007/978-3-319-21401-6_26
- Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *Formal Methods and Software Engineering - 23rd International Conference on Formal Engineering Methods, ICFEM 2022, Madrid, Spain, October 24-27, 2022, Proceedings (Lecture Notes in Computer Science, Vol. 13478)*, Adrián Riesco and Min Zhang (Eds.). Springer, 90–105. doi:10.1007/978-3-031-17244-1_6
- Dan Doel and Erik de Castro Lopo. 2025. *vector-algorithms: Efficient algorithms for vector arrays*. <https://hackage.haskell.org/package/vector-algorithms>
- Manuel Fähndrich and Robert DeLine. 2002. Adoption and Focus: Practical Linear Types for Imperative Programming. In *Proceedings of the 2002 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Berlin, Germany, June 17-19, 2002, Jens Knoop and Laurie J. Hendren (Eds.). ACM, 13–24. doi:10.1145/512529.512532
- Matthew Fluet and Greg Morrisett. 2004. Monadic Regions. In *Proceedings of the Ninth ACM SIGPLAN International Conference on Functional Programming, ICFP 2004, Snow Bird, UT, USA, September 19-21, 2004*, Chris Okasaki and Kathleen Fisher (Eds.). ACM, 103–114. doi:10.1145/1016850.1016867
- Matthew Fluet and Greg Morrisett. 2006. Monadic regions. *J. Funct. Program.* 16, 4-5 (2006), 485–545. doi:10.1017/S095679680600596X
- Matthew Fluet, Greg Morrisett, and Amal J. Ahmed. 2006. Linear Regions Are All You Need. In *Programming Languages and Systems, 15th European Symposium on Programming, ESOP 2006, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2006, Vienna, Austria, March 27-28, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 3924)*, Peter Sestoft (Ed.). Springer, 7–21. doi:10.1007/11693024_2
- Lennard Gäher, Michael Sammler, Ralf Jung, Robbert Krebbers, and Derek Dreyer. 2024. RefinedRust: A Type System for High-Assurance Verification of Rust Programs. *Proc. ACM Program. Lang.* 8, PLDI (2024), 1115–1139. doi:10.1145/3656422

- Aïna Linn Georges, Benjamin Peters, Laila Elbeheiry, Leo White, Stephen Dolan, Richard A. Eisenberg, Chris Casinghino, François Pottier, and Derek Dreyer. 2025. Data Race Freedom à la Mode. *Proc. ACM Program. Lang.* 9, POPL (2025), 656–686. doi:10.1145/3704859
- Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4
- Dan Grossman, Greg Morrisett, Trevor Jim, Michael W. Hicks, Yanling Wang, and James Cheney. 2002. Region-Based Memory Management in Cyclone. In *Proceedings of the 2002 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), Berlin, Germany, June 17-19, 2002*, Jens Knoop and Laurie J. Hendren (Eds.). ACM, 282–293. doi:10.1145/512529.512563
- Philipp Haller and Martin Odersky. 2010. Capabilities for Uniqueness and Borrowing. In *ECOOP 2010 - Object-Oriented Programming, 24th European Conference, Maribor, Slovenia, June 21-25, 2010. Proceedings (Lecture Notes in Computer Science, Vol. 6183)*, Theo D'Hondt (Ed.). Springer, 354–378. doi:10.1007/978-3-642-14107-2_17
- Son Ho. 2024. *Formal Verification of Rust Programs by Functional Translation. (Vérification formelle de programmes Rust par traduction fonctionnelle)*. Ph.D. Dissertation. PSL University, France. <https://tel.archives-ouvertes.fr/tel-05004605>
- Son Ho and Jonathan Protzenko. 2022. AENEAS: Rust Verification by Functional Translation. *Proc. ACM Program. Lang.* 6, ICFP (2022), 711–741. doi:10.1145/3547647
- C. A. R. Hoare. 1961. Algorithm 64: Quicksort. *Commun. ACM* 4, 7 (1961), 321. doi:10.1145/366622.366644
- Paul Hudak, Simon L. Peyton Jones, Philip Wadler, Brian Boutel, Jon Fairbairn, Joseph H. Fasel, María M. Guzmán, Kevin Hammond, John Hughes, Thomas Johnsson, Richard B. Kieburtz, Rishiyur S. Nikhil, Will Partain, and John Peterson. 1992. Report on the Programming Language Haskell, A Non-strict, Purely Functional Language. *ACM SIGPLAN Notices* 27, 5 (1992), 1. doi:10.1145/130697.130699
- Jane Street. 2025. *OxCaml*. <https://oxcaml.org/>
- Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018a. RustBelt: Securing the Foundations of the Rust Programming Language. *Proc. ACM Program. Lang.* 2, POPL (2018), 66:1–66:34. doi:10.1145/3158154
- Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018b. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. doi:10.1017/S0956796818000151
- Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*, Sriram K. Rajamani and David Walker (Eds.). ACM, 637–650. doi:10.1145/2676726.2676980
- Oleg Kiselyov and Chung-chieh Shan. 2008. Lightweight Monadic Regions. In *Proceedings of the 1st ACM SIGPLAN Symposium on Haskell, Haskell 2008, Victoria, BC, Canada, 25 September 2008*, Andy Gill (Ed.). ACM, 1–12. doi:10.1145/1411286.1411288
- John Launchbury. 1993. A Natural Semantics for Lazy Evaluation. In *Conference Record of the Twentieth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Charleston, South Carolina, USA, January 1993*, Mary S. Van Deusen and Bernard Lang (Eds.). ACM Press, 144–154. doi:10.1145/158511.158618
- John Launchbury and Simon L. Peyton Jones. 1994. Lazy Functional State Threads. In *Proceedings of the ACM SIGPLAN'94 Conference on Programming Language Design and Implementation (PLDI), Orlando, Florida, USA, June 20-24, 1994*, Vivek Sarkar, Barbara G. Ryder, and Mary Lou Soffa (Eds.). ACM, 24–35. doi:10.1145/178243.178246
- Andrew Lelechenko. 2021. *tasty-bench: Featherlight benchmark framework*. <https://hackage.haskell.org/package/tasty-bench>
- Anton Lorenzen, Leo White, Stephen Dolan, Richard A. Eisenberg, and Sam Lindley. 2024. Oxidizing OCaml with Modal Memory Management. *Proc. ACM Program. Lang.* 8, ICFP (2024), 485–514. doi:10.1145/3674642
- Danielle Marshall, Michael Vollmer, and Dominic Orchard. 2022. Linearity and Uniqueness: An Entente Cordiale. In *Programming Languages and Systems - 31st European Symposium on Programming, ESOP 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings (Lecture Notes in Computer Science, Vol. 13240)*, Ilya Sergey (Ed.). Springer, 346–375. doi:10.1007/978-3-030-99336-8_13
- Nicholas D. Matsakis. 2025. *Move, Destruct, Forget, and Rust*. <https://smallcultfollowing.com/babysteps/blog/2025/10/21/move-destruct-leak/>
- Nicholas D. Matsakis and Felix S. Klock, II. 2014. The Rust Language. In *Proceedings of the 2014 ACM SIGAda annual conference on High integrity language technology, HILT 2014, Portland, Oregon, USA, October 18-21, 2014*, Michael B. Feldman and S. Tucker Taft (Eds.). ACM, 103–104. doi:10.1145/2663171.2663188
- Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code. In *PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*, Ranjit Jhala and Isil Dillig (Eds.). ACM, 841–856. doi:10.1145/3519939.3523704
- Yusuke Matsushita and Hiromi Ishii. 2026a. *Artifact for PLDI 2026 "Pure Borrow: Linear Haskell Meets Rust-Style Borrowing"*. doi:10.5281/zenodo.19622061

- Yusuke Matsushita and Hiromi Ishii. 2026b. Pure Borrow: Linear Haskell Meets Rust-Style Borrowing. *Proc. ACM Program. Lang.* 10, PLDI (2026). doi:10.1145/3808259
- Yusuke Matsushita and Takeshi Tsukada. 2025. Nola: Later-Free Ghost State for Verifying Termination in Iris. *Proc. ACM Program. Lang.* 9, PLDI (2025), 98–124. doi:10.1145/3729250
- Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-based Verification for Rust Programs. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, 484–514. doi:10.1007/978-3-030-44914-8_18
- Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2021. RustHorn: CHC-based Verification for Rust Programs. *ACM Trans. Program. Lang. Syst.* 43, 4 (2021), 15:1–15:54. doi:10.1145/3462205
- Yasuhiko Minamide. 1998. A Functional Representation of Data Structures with a Hole. In *POPL '98, Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Diego, CA, USA, January 19-21, 1998*, David B. MacQueen and Luca Cardelli (Eds.). ACM, 75–84. doi:10.1145/268946.268953
- David R. Musser. 1997. Introspective Sorting and Selection Algorithms. *Softw. Pract. Exp.* 27, 8 (1997), 983–993.
- Takashi Nakayama, Yusuke Matsushita, Ken Sakayori, Ryosuke Sato, and Naoki Kobayashi. 2024. Borrowable Fractional Ownership Types for Verification. In *Verification, Model Checking, and Abstract Interpretation - 25th International Conference, VMCAI 2024, London, United Kingdom, January 15-16, 2024, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 14500)*, Rayna Dimitrova, Ori Lahav, and Sebastian Wolff (Eds.). Springer, 224–246. doi:10.1007/978-3-031-50521-8_11
- Simon L. Peyton Jones and Philip Wadler. 1993. Imperative Functional Programming. In *Conference Record of the Twentieth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Charleston, South Carolina, USA, January 1993*, Mary S. Van Deusen and Bernard Lang (Eds.). ACM Press, 71–84. doi:10.1145/158511.158524
- Andrew M. Pitts and Ian David Bede Stark. 1993. Observable Properties of Higher Order Functions that Dynamically Create Local Names, or: What's new?. In *Mathematical Foundations of Computer Science 1993, 18th International Symposium, MFCS'93, Gdansk, Poland, August 30 - September 3, 1993, Proceedings (Lecture Notes in Computer Science, Vol. 711)*, Andrzej M. Borzyszkowski and Stefan Sokolowski (Eds.). Springer, 122–141. doi:10.1007/3-540-57182-5_8
- Arnaud Spiwack, Csongor Kiss, Jean-Philippe Bernardy, Nicolas Wu, and Richard A. Eisenberg. 2022. Linearly Qualified Types: Generic Inference for Capabilities and Uniqueness. *Proc. ACM Program. Lang.* 6, ICFP (2022), 137–164. doi:10.1145/3547626
- Nikhil Swamy, Catalin Hritcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean Karim Zinzindohoue, and Santiago Zanella-Béguelin. 2016. Dependent Types and Multi-monadic Effects in F*. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, Rastislav Bodík and Rupak Majumdar (Eds.). ACM, 256–270. doi:10.1145/2837614.2837655
- The Rust Community. 2017. 2094-nll - The Rust RFC Book. <https://rust-lang.github.io/rfcs/2094-nll.html>
- Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. 2024. A Logical Approach to Type Soundness. *J. ACM* 71, 6 (2024), 40:1–40:75. doi:10.1145/3676954
- Mads Tofte and Jean-Pierre Talpin. 1994. Implementation of the Typed Call-by-Value λ -calculus using a Stack of Regions. In *Conference Record of POPL '94: 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Portland, Oregon, USA, January 17-21, 1994*, Hans-Juergen Boehm, Bernard Lang, and Daniel M. Yellin (Eds.). ACM Press, 188–201. doi:10.1145/174675.177855
- David N. Turner, Philip Wadler, and Christian Mossin. 1995. Once Upon a Type. In *Proceedings of the seventh international conference on Functional programming languages and computer architecture, FPCA 1995, La Jolla, California, USA, June 25-28, 1995*, John Williams (Ed.). ACM, 1–11. doi:10.1145/224164.224168
- Sebastian Ullrich. 2016. *Simple Verification of Rust Programs via Functional Purification*. Master's thesis. Karlsruhe Institute of Technology.
- Philip Wadler. 1990. Linear Types can Change the World!. In *Programming concepts and methods: Proceedings of the IFIP Working Group 2.2, 2.3 Working Conference on Programming Concepts and Methods, Sea of Galilee, Israel, 2-5 April, 1990*, Manfred Broy and Cliff B. Jones (Eds.). North-Holland, 561.
- Andrew Wagner, Olek Gierczak, Brianna Marshall, John M. Li, and Amal Ahmed. 2025. From Linearity to Borrowing. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 415 (Oct. 2025), 27 pages. doi:10.1145/3764117
- Keith Wansbrough and Simon L. Peyton Jones. 1999. Once Upon a Polymorphic Type. In *POPL '99, Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Antonio, TX, USA, January 20-22, 1999*, Andrew W. Appel and Alex Aiken (Eds.). ACM, 15–28. doi:10.1145/292540.292545
- Andrew K. Wright and Matthias Felleisen. 1994. A Syntactic Approach to Type Soundness. *Inf. Comput.* 115, 1 (1994), 38–94. doi:10.1006/INCO.1994.1093
- Joshua Yanovski, Hoang-Hai Dang, Ralf Jung, and Derek Dreyer. 2021. GhostCell: Separating Permissions from Data in Rust. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–30. doi:10.1145/3473597

$$\begin{array}{c}
T \preccurlyeq T \quad \frac{T \preccurlyeq T' \quad T' \preccurlyeq T''}{T \preccurlyeq T''} \quad (\forall x. T) \preccurlyeq T[a/x] \quad \frac{x \text{ is fresh in } U \quad U \preccurlyeq T}{U \preccurlyeq \forall x. T} \\
\frac{T' \preccurlyeq T \quad U \preccurlyeq U' \quad \pi \leq \mu}{T \rightarrow_\pi U \preccurlyeq T' \rightarrow_\mu U'} \quad \frac{\forall i. T_i[\overline{U}/X] \preccurlyeq T_i[\overline{U'}/X] \quad \text{data } F \bar{X} \text{ where } C :: \overline{T} \rightarrow_\pi F \bar{X}}{F \bar{U} \preccurlyeq F \bar{U'}} \text{ SUBTY-DATA} \\
\frac{T \preccurlyeq U}{\text{Ref } T \preccurlyeq \text{Ref } U} \quad \frac{\beta \leq \alpha}{\text{End}^\alpha \preccurlyeq \text{End}^\beta} \quad \frac{\beta \leq \alpha \quad T \preccurlyeq U \quad U \preccurlyeq T}{\text{Mut}^\alpha T \preccurlyeq \text{Mut}^\beta U} \\
\frac{\beta \leq \alpha \quad T \preccurlyeq U}{\text{Share}^\alpha T \preccurlyeq \text{Share}^\beta U} \quad \frac{\alpha \leq \beta \quad T \preccurlyeq U}{\text{Lend}^\alpha T \preccurlyeq \text{Lend}^\beta U} \quad \frac{\beta \leq \alpha \quad T \preccurlyeq U}{\text{BO}^\alpha T \preccurlyeq \text{BO}^\beta U}
\end{array}$$

Fig. 25. Subtyping rules.

$$\Gamma \preccurlyeq \Gamma \quad \frac{\Gamma \preccurlyeq \Gamma' \quad \Gamma' \preccurlyeq \Gamma''}{\Gamma \preccurlyeq \Gamma''} \quad \frac{\mu \leq \pi \quad T \preccurlyeq U}{x ::_\pi T, \Gamma \preccurlyeq x ::_\mu U, \Gamma} \quad x ::_\omega T, \Gamma \preccurlyeq \Gamma$$

Fig. 26. Rules for typing context inclusion.

A Formal Calculus

We present the omitted details of our formal calculus introduced in §5.

A.1 Type System

We present the omitted details of our type system introduced in §5.2.

Basics. The typing context Γ consists of items of the form $x ::_\pi T$, with the multiplicity π .²⁸

We define the multiplicity product $\pi \cdot \mu$ naturally, i.e., $1 \cdot \pi \triangleq \pi \cdot 1 \triangleq \pi$, $\omega \cdot \pi \triangleq \pi \cdot \omega \triangleq \omega$, $\prod \bar{\pi} \cdot \prod \bar{\mu} \triangleq \prod (\bar{\pi}, \bar{\mu})$. This is introduced to define the typing context multiplication $\pi \Gamma$ (Fig. 27). The formal product of variables $\prod \bar{\pi}$ is introduced for the multiplicity product.²⁹

Inclusion and subtyping. The lifetime inclusion $\alpha \leq \beta$ is the partial order over lifetimes generated by the rules $\alpha \leq \text{static}$, $\alpha_0 \wedge \alpha_1 \leq \alpha_i$ and “if $\beta \leq \alpha_0$ and $\beta \leq \alpha_1$, then $\beta \leq \alpha_0 \wedge \alpha_1$ ”. The multiplicity inclusion $\pi \leq \mu$ is the partial order over multiplicities generated by the rules $1 \leq \pi$, $\pi \leq \omega$, $\pi_i \leq \pi_0 \cdot \pi_1$, and “if $\pi_0 \leq \mu$ and $\pi_1 \leq \mu$, then $\pi_0 \cdot \pi_1 \leq \mu$ ”.

Our type system introduces subtyping natively, without a function like upcast (Fig. 13, §3.2) in our Haskell implementation. The rules for the subtyping $T \preccurlyeq U$ are listed in Fig. 25. To be precise, we use mixed induction (i.e., coinduction over induction) to reason about recursive data types, following the approach of Danielsson and Altenkirch [2010]. More specifically, the subtyping rule SUBTY-DATA for the user-defined data type F can be applied coinductively, whereas the other rules can only be applied inductively. Overall, the subtyping relation is defined as the greatest fixpoint over the least fixpoint.

We also introduce the typing context inclusion $\Gamma \preccurlyeq \Delta$, which is inductively defined by the rules listed in Fig. 26. We can weaken the type of a variable by subtyping. A variable of the unrestricted multiplicity ω can be forgotten and also turned into the multiplicity 1.

Operations on typing contexts. Following Bernardy et al. [2018], we introduce the multiplication $\pi \Gamma$ and the sum $\Gamma + \Delta$ over typing contexts, defined in Fig. 27. Intuitively, $\Gamma + \Delta$ allows sharing unrestricted variables between Γ and Δ .

²⁸ The order of items in a typing context Γ is ignored. For each variable x , a typing context Γ can contain at most one item of the form $x ::_\pi T$. We write $\emptyset$ for the empty typing context. The domain $\text{dom } \Gamma$ is defined as $\{x \mid x ::_\pi T \in \Gamma\}$.

²⁹ In $\prod \bar{\pi}$, the sequence $\bar{\pi}$ should be non-empty, and its order is ignored. The substitution $(\prod \bar{\pi})[\bar{\mu}/\bar{\mu}]$ is defined by the multiplicity product.

$$\begin{aligned}
\pi \emptyset &\triangleq \emptyset & \pi(x ::_{\mu} T, \Gamma) &\triangleq x ::_{\pi \cdot \mu} T, \pi \Gamma & \emptyset + \Delta &\triangleq \Delta \\
(x ::_{\omega} T, \Gamma) + (x ::_{\omega} T, \Delta) &\triangleq x ::_{\omega} T, (\Gamma + \Delta) & \frac{x \notin \text{dom } \Delta}{(x ::_{\pi} T, \Gamma) + \Delta} &\triangleq x ::_{\pi} T, (\Gamma + \Delta)
\end{aligned}$$

Fig. 27. Operations on typing contexts.

data () **where** () :: () **data** Bool **where** True :: Bool, False :: Bool
data Ur X **where** Ur :: X → Ur X **data** (X, Y) **where** (,) :: X → Y → (X, Y)

Fig. 28. Default data type declarations.

$$\begin{aligned}
&\frac{\Delta \preccurlyeq \Gamma \quad \Gamma \vdash t :: T}{\Delta \vdash t :: T} & \frac{\Gamma \vdash t :: T \quad T \preccurlyeq U}{\Gamma \vdash t :: U} & \frac{x \text{ is fresh in } \Gamma \quad \Gamma \vdash t :: T}{\Gamma \vdash t :: \forall x. T} & x ::_{\pi} T \vdash x :: T \\
&\frac{\forall i. \Gamma_i \vdash t :: T_i \quad \overline{x ::_{\pi} T}, \Delta \vdash u :: U}{\sum_i \pi \Gamma_i + \Delta \vdash \text{let } \bar{x} = \bar{t} \text{ in } u :: U} & \frac{\forall i. \overline{x ::_{\omega} T}, \Gamma \vdash t_i :: T_i \quad \overline{x ::_{\omega} T}, \Delta \vdash u :: U}{\omega \Gamma + \Delta \vdash \text{let } \bar{x} = \bar{t} \text{ in } u :: U} \\
&\frac{x ::_{\pi} T, \Gamma \vdash t :: U}{\Gamma \vdash \lambda x \rightarrow t :: T \rightarrow_{\pi} U} & \frac{\Gamma \vdash t :: T \rightarrow_{\pi} U \quad \Delta \vdash u :: T}{\Gamma + \pi \Delta \vdash t u :: U} & \frac{x \in \text{dom } \Gamma \quad \Gamma \vdash t :: T}{\Gamma \vdash \text{seq } x t :: T} \\
&\vdash n :: \text{Int} & \frac{\forall i. \Gamma_i \vdash t_i :: T_{j,i}[\overline{U/X}] \quad \text{data } F \bar{X} \text{ where } \overline{C :: \overline{T} \rightarrow_{\pi} F \bar{X}}}{\sum_i \pi_{j,i} \Gamma_i \vdash C_j \bar{t} :: F \bar{U}} \\
&\frac{\Gamma \vdash t :: F \bar{U}}{\forall j. x ::_{\pi_j} T_j[\overline{U/X}], \Delta \vdash u_j :: U'} & \frac{\Gamma \vdash t :: B^{\alpha}(F \bar{U})}{\forall j. x ::_{\pi_j} B^{\alpha}(T_j[\overline{U/X}]), \Delta \vdash u_j :: U'} \\
&\frac{\text{data } F \bar{X} \text{ where } \overline{C :: \overline{T} \rightarrow_{\pi} F \bar{X}}}{\Gamma + \Delta \vdash \text{case } t \text{ of } \{\overline{C \bar{x} \rightarrow u}\} :: U'} & \frac{\text{data } F \bar{X} \text{ where } \overline{C :: \overline{T} \rightarrow_{\pi} F \bar{X}}}{\Gamma + \Delta \vdash \text{case } t \text{ of } \{\overline{C \bar{x} \rightarrow u}\} :: U'} & \text{TY-CASE-BOR} \\
&\frac{a :: \bar{T}; U \quad \forall i. \Gamma_i \vdash t_i :: T_i}{\sum_i \Gamma_i \vdash a \bar{t} :: U} & iop :: \overline{\text{Int}}; \text{Int} & irel :: \overline{\text{Int}}; \text{Bool} & \text{par} :: T, U; (T, U) \\
&\frac{T \text{ is Linearly or Mut}^{\alpha} U}{\text{consume} :: T; ()} & \frac{T \text{ is Int, End}^{\alpha} \text{ or Share}^{\alpha} U}{\text{move} :: T; \text{Ur } T} & \text{linearly} :: \text{Linearly} \multimap \text{Ur } T; \text{Ur } T \\
&\frac{T \text{ is Linearly, Ref } U, \text{Now}^{\alpha} \text{ or Mut}^{\alpha} U}{\text{withLinearly} :: T; (\text{Linearly}, T)} & \text{newRef} :: T; \text{Ref } T & \text{freeRef} :: \text{Ref } T; T \\
&\text{newLifetime} :: (\forall \iota. \text{Now}^{\alpha \iota} \multimap T); T & \text{endLifetime} :: \text{Now}^{\alpha \iota}; \text{End}^{\alpha \iota} \\
&\text{borrow} :: \text{Linearly}, T; (\text{Mut}^{\alpha} T, \text{Lend}^{\alpha} T) & \text{share} :: \text{Mut}^{\alpha} T; \text{Ur } (\text{Share}^{\alpha} T) \\
&\frac{T \text{ is Int, End}^{\beta} \text{ or Share}^{\beta} U}{\text{copy} :: \text{Share}^{\alpha} T; T} & \text{joinMut} :: B^{\alpha}(\text{Mut}^{\beta} T); B^{\alpha \wedge \beta} T & \text{reclaim} :: \text{Lend}^{\alpha}, T \text{End}^{\alpha}; T \\
&\text{execBO} :: \text{Now}^{\alpha}, \text{BO}^{\alpha} T; (\text{Now}^{\alpha}, T) & \text{pure} :: T; \text{BO}^{\alpha} T & (\gg) :: \text{BO}^{\alpha} T, T \multimap \text{BO}^{\alpha} U; \text{BO}^{\alpha} U \\
&\text{sexecBO} :: \text{Now}^{\alpha}, \text{BO}^{\alpha \wedge \beta} U; \text{BO}^{\beta}(\text{Now}^{\alpha}, U) & \text{parBO} :: \text{BO}^{\alpha} T, \text{BO}^{\alpha} U; \text{BO}^{\alpha}(T, U) \\
&\text{deref} :: B^{\alpha}(\text{Ref } T); \text{BO}^{\alpha}(B^{\alpha} T) & \frac{\beta \leq \alpha}{\text{updateRef} :: T \multimap \text{BO}^{\beta}(U, T), \text{Mut}^{\alpha}(\text{Ref } T); \text{BO}^{\beta}(U, \text{Mut}^{\alpha}(\text{Ref } T))}
\end{aligned}$$

Fig. 29. Typing rules.

Typing rules. The typing rules are listed in Fig. 29. For this, we assume the default data type declarations listed in Fig. 28. The typing judgment has the form $\Gamma \vdash t :: T$. For operators op and monad constructors mo , we introduce an auxiliary judgment $a :: \bar{T}; U$ (where a is either op or mo)

Denesting context $N ::= [\cdot] t \mid x [\cdot] \mid \mathbf{seq} \ x [\cdot] \mid C \bar{x} [\cdot] \bar{t}$
 $\mid \mathbf{case} [\cdot] \mathbf{of} \{ \bar{C} \bar{x} \rightarrow t \} \mid op \ \bar{x} [\cdot] \bar{t} \mid mo \ \bar{x} [\cdot] \bar{t}$
 (a) Common things.

Extra operator $\hat{op} ::= \mathbf{linear} \ 1 \mid \mathbf{exeBO} \ 1$ $\mid \mathbf{execBO}^\circ \ 1 \mid (\gg^\circ) \ 2 \mid \mathbf{sexecBO}^+ \ 2$ $\mid \mathbf{sexecBO}^\circ \ 1 \mid \mathbf{parBO}^\circ \ 2 \mid \mathbf{deref}^\circ \ 1$ $\mid \mathbf{updateRef}^+ \ 2 \mid \mathbf{updateRef}_\ell \ 1$ $\mid \mathbf{updateRef}_\ell^\circ \ 1$ (b) For mutative semantics.	Borrower context $\hat{B} ::= [\cdot] \mid \hat{B} \hat{B}$ Extra operator $\hat{op} ::= \mathbf{linear} \ 1 \mid \mathbf{exeBO}_H \ 1$ $\mid \mathbf{execBO}^\circ \ 1 \mid (\gg^\circ) \ 2 \mid \mathbf{sexecBO}_H^+ \ 2$ $\mid \mathbf{sexecBO}_{H;H'}^\circ \ 1 \mid \mathbf{parBO}_H^\circ \ 2 \mid \mathbf{deref}_H^\circ \ 1$ $\mid \mathbf{updateRef}_H^+ \ 2 \mid \mathbf{updateRef}_{\bar{p}} \ 1$ $\mid \mathbf{updateRef}_{\bar{p};H}^\circ \ 1$ (c) For denotational semantics.
--	---

Fig. 30. More syntax for the two operational semantics.

and the length of $\bar{T}$ is the arity of a), meaning that a linearly inputs arguments of the type $\bar{T}$ and outputs an object of the type U .

We have two typing rules for typing the **case** term. The latter **TY-CASE-BOR** is for the case where the pattern-matched term is a borrower $B^\alpha (F \bar{U})$. We distribute the borrower constructor B^α to the components $\bar{x}$ of the data. This is an idealized version of the API functions such as **splitPair** (Fig. 12, §3.2) in our Haskell implementation.

A.2 Two Operational Semantics

We present the omitted details of our two operational semantics introduced in §5.3.

Basics. In denotational operational semantics, we use a borrower constructor $\hat{B}$, where a mutable borrower $\mathbf{Mut}^{\bar{p}}$ is marked with borrow paths $\bar{p}$, representing which part of which lenders the borrower refers to. Here, we can have multiple borrow paths due to the reborrowing caused by **joinMut**, as clarified by the reduction rule **DRED-JOIN-BOR** shown later in Fig. 36b. We use a borrower context $\hat{B}$ for a composite of borrower constructors. We write $\hat{B} \ a$ instead of $\hat{B}[a]$ for readability.

In both semantics, an environment $\hat{E}, \hat{E}$ assigns a term $\hat{t}, t$ to each variable x . A denotational environment $\hat{E}$ additionally manages the borrow ids $\bar{\delta}$ that have been created.³⁰ The memory $\bar{M}$ appears only in mutative semantics.³¹ A configuration has the form $\hat{E}; \bar{M}; x$ in mutative semantics and $\hat{E}; x$ in denotational semantics, where x is the top-level variable.³² We say that $\hat{E}; \bar{M}; x$ is a normal form if $\hat{E}$ has an item of the form $x = \hat{v}$. Likewise, we say that $\hat{E}; x$ is a normal form if $\hat{E}$ has an item of the form $x = \hat{v}$.

Strictness works as follows in this language. For simplicity, all the arguments of operators are made strict. In particular, **par** $t \ u$ forces the evaluation of t and u before returning the pair of their results. Also, we introduce a special term **seq** $x \ t$ that forces the evaluation of the variable x before returning t ; it amounts to **do** $!x \leftarrow x; t$ in Haskell, and we introduce **seq** as a primitive for the bang ! pattern. The usual function application is lazy. Pattern matching of **case** is strict.

³⁰ The order of items in an environment $\hat{E}, \hat{E}$ is ignored. An environment $\hat{E}, \hat{E}$ can have at most one item of the form $x = a$ for each variable x . Also, a denotational environment $\hat{E}$ can have at most one occurrence of each borrow id $\bar{\delta}$.

³¹ The order of items in a memory $\bar{M}$ is ignored. A memory $\bar{M}$ can have at most one item of the form $\ell \mapsto x$ for each location ℓ .

³² We assume alpha-equivalence over mutative configurations $\hat{E}; \bar{M}; x$ with respect to variable names x bound in the environment $\hat{E}$, and assume alpha-equivalence over denotational configurations $\hat{E}; x$ with respect to variable names x and borrow id names $\bar{\delta}$ bound in the environment $\hat{E}$.

$$\begin{array}{c}
\frac{\text{dom } H \cap \text{dom } H' = \emptyset}{H \cdot H' \triangleq H, H'} \quad \frac{(p \leftarrow x, H) / (p \leftarrow y, H') \triangleq p \leftarrow y, (H / H')}{\frac{p \notin \text{dom } H'}{(p \leftarrow x, H) / H' \triangleq p \leftarrow x, (H / H')}} \quad \emptyset / H \triangleq H
\end{array}$$

Fig. 31. Parallel $H \cdot H'$ and sequential H / H' composition of histories.

$$\begin{array}{c}
\frac{x = \hat{K}[y] \in \hat{E} \quad \hat{E}; M; y \text{ loop}}{\hat{E}; M; x \text{ loop}} \quad \frac{x = \dot{K}[y] \in \dot{E} \quad \dot{E}; y \text{ loop}}{\dot{E}; x \text{ loop}}
\end{array}$$

Fig. 32. The coinductive rule of the forcing loop predicate $\hat{G}$ loop, $\dot{G}$ loop for each semantics.

Figure 30 shows additional syntax for the two operational semantics, omitted in Fig. 21.

The denesting context N is used for turning a nested term into an unnested one, by binding subterms to fresh variables (see **MRED-DENEST** and **DRED-DENEST** later shown in Fig. 33).³³ Denesting is a dynamic version of conversion to the A-normal (or K-normal) form.

We introduce extra values $\hat{d}, \dot{d}$ and extra operators $\hat{op}, \dot{op}$ for the two semantics. For monadic extra operators such as **execBO**^o, the superscript o indicates post-processing, $+$ pre-processing, and $\bullet$ pre-processing before post-processing. The value $\emptyset / \emptyset_H$ represents the value of linearity witness tokens Linearly and lifetime tokens **Now**^α, **End**^α; in denotational semantics, the lifetime tokens carry the history H .

Composing histories. We define the parallel $H \cdot H'$ and sequential H / H' compositions of histories, as defined in Fig. 31. The parallel composition $H \cdot H'$ treats both histories equally (and thus is commutative), while the sequential composition H / H' overwrites older records H with newer ones H' . The domain $\text{dom } H$ is defined as $\{p \mid p \leftarrow x \in H\}$.

Reduction rules. Now we inductively define the small-step reduction relations $\hat{G} \rightarrow \hat{G}'$ and $\dot{G} \rightarrow \dot{G}'$ for mutative and denotational operational semantics, respectively.

The basic reduction rules for the two semantics are listed in Fig. 33.³⁴ The rules **MRED-LOOP** and **DRED-LOOP** use the forcing loop predicate $\hat{G}$ loop, $\dot{G}$ loop, meaning that the environment contains variables that circularly forcing one another, *coinductively* defined by the rules in Fig. 32. The two semantics are mostly the same in these rules, but there are some notable differences due to borrower constructors $\hat{B}$, appearing only in denotational semantics. The rule **DRED-VAR** for a variable term carries the borrower context $\hat{B}$. Also, the rule **DRED-CASE-BOR** distributes the borrower constructor $\hat{B}$ over the data components (recall the typing rule **TY-CASE-BOR** in Fig. 29).

The reduction rules for basic operators for denotational semantics are listed in Fig. 34. The rules for mutative semantics are just analogous to them. The operator linearly allows binding the result of a linear computation to an unrestricted variable. We use an extra operator linear to clarify that boundary for our association system **Appendix B**.

The basic reference-related reduction rules for the two semantics are presented in Fig. 35. In mutative semantics, a reference is modeled as **Ref** ℓ with a location ℓ , whose content x is separately stored as $\ell \mapsto x$ in the global memory M ; in denotational semantics, a reference is modeled as **Ref** x , where the body content x is locally stored in the value. The ownership justifies this.

The borrow-related reduction rules for the two semantics are listed in Fig. 36. Here we clearly see the differences between the two semantics. In denotational semantics, for each new borrow, a fresh borrow id δ is created, and the lender is modeled as **Lend**^δ x , where the x represents the original

³³ We impose left-to-right order in denesting to avoid unnecessary nondeterminism.

³⁴ The function `asBool` maps the truth and falsehood to `True` and `False`, respectively.

$$\begin{array}{c}
\frac{x = \dot{K}[y] \in \hat{E} \quad \hat{E}; M; y \rightarrow \hat{E}'; M'; y}{\hat{E}; M; x \rightarrow \hat{E}'; M'; x} \text{MRED-KTX} \quad \frac{\hat{G} \text{ loop}}{\hat{G} \rightarrow \hat{G}} \text{MRED-LOOP} \\
x = \text{let } \overline{y = t} \text{ in } u, \hat{E}; M; x \rightarrow \quad \frac{t \notin \text{Var}}{x = N[t], \hat{E}; M; x \rightarrow y = t, x = N[y], \hat{E}; M; x} \text{MRED-DENEST} \\
\frac{y = \hat{v} \in \hat{E}}{x = y, \hat{E}; M; x \rightarrow x = \hat{v}, \hat{E}; M; x} \text{MRED-VAR} \quad \frac{f = \lambda y \rightarrow t \in \hat{E}}{x = f y, \hat{E}; M; x \rightarrow x = t, \hat{E}; M; x} \\
\frac{y = \hat{v} \in \hat{E}}{x = \text{seq } y z, \hat{E}; M; x \rightarrow x = z, \hat{E}; M; x} \quad \frac{y = C_i \overline{z'} \in \hat{E}}{x = \text{case } y \text{ of } \{ \overline{C \bar{z} \rightarrow t} \}, \hat{E}; M; x \rightarrow \overline{z_i = z'}, x = t_i, \hat{E}; M; x}
\end{array}$$

(a) For mutative operational semantics.

$$\begin{array}{c}
\frac{x = \dot{K}[y] \in \dot{E} \quad \dot{E}; y \rightarrow \dot{E}'; y}{\dot{E}; x \rightarrow \dot{E}'; x} \text{DRED-KTX} \quad \frac{\dot{G} \text{ loop}}{\dot{G} \rightarrow \dot{G}} \text{DRED-LOOP} \\
x = \text{let } \overline{y = t} \text{ in } u, \dot{E}; x \rightarrow \quad \frac{t \notin \text{Var}}{x = N[t], \dot{E}; x \rightarrow y = t, x = N[y], \dot{E}; x} \text{DRED-DENEST} \\
\frac{y = \dot{v} \in \dot{E}}{x = \dot{B} y, \dot{E}; x \rightarrow x = \dot{B} \dot{v}, \dot{E}; x} \text{DRED-VAR} \quad \frac{f = \lambda y \rightarrow t \in \dot{E}}{x = f y, \dot{E}; x \rightarrow x = t, \dot{E}; x} \\
\frac{y = \dot{v} \in \dot{E}}{x = \text{seq } y z, \dot{E}; x \rightarrow x = z, \dot{E}; x} \quad \frac{y = C_i \overline{z'} \in \dot{E}}{x = \text{case } y \text{ of } \{ \overline{C \bar{z} \rightarrow t} \}, \dot{E}; x \rightarrow \overline{z_i = z'}, x = t_i, \dot{E}; x} \\
\frac{y = \dot{B}(C_i \overline{z'}) \in \dot{E} \quad \forall j. \dot{B}'_j = \begin{cases} \text{Mut}^{\dot{p}, j} & \dot{B} = \text{Mut}^{\dot{p}} \\ \text{Share} & \dot{B} = \text{Share} \end{cases}}{x = \text{case } y \text{ of } \{ \overline{C \bar{z} \rightarrow t} \}, \dot{E}; x \rightarrow \overline{z_i = \dot{B}'_i z'}, x = t_i, \dot{E}; x} \text{DRED-CASE-BOR}
\end{array}$$

(b) For denotational operational semantics.

Fig. 33. Basic reduction rules.

version of the borrowed object. Notably, when the lender reclaims the ownership of the borrowed object by **DRED-RECLAIM**, the latest version of the object is restored from the history. For this, we introduce the restoration-by-history predicate $\text{restore}_H p \dot{E} \overset{x \rightsquigarrow y}{\rightarrow} \dot{E}'$, inductively defined by the rules listed in Fig. 36c. The predicate adds new items to the environment $\dot{E}$ to get $\dot{E}'$, reflecting updates recorded in the history H on the borrow path p for the variable x , returning a new variable x' . The predicate does nothing and just returns $\dot{E}$ and x if the history does not contain any relevant update, while the predicate takes a fresh variable x' for the new version of x otherwise.

Basic monadic reduction rules for denotational semantics are listed in Fig. 37. The rules for mutative semantics are just analogous to them, just without the history information H . We internally use the extra operator **exeBO/exeBO_H**, a utility variant of **execBO**, to execute a monad without carrying around the live lifetime token. It returns **Done** x /**Done_H** x for the result of the computation. What is important is how to handle histories. The scoped execution **sexecBO** executes the monad bo with the empty history, whose resulting history H_0 is then sequentially composed with the original histories H, H' . The parallel execution **parBO** executes the monads bo_0, bo_1 respectively with the empty history and then composes the resulting histories H_0, H_1 in parallel, whose composite is sequentially composed with the original history H .

Reference-related monadic reduction rules for the two semantics are listed in Fig. 38. Like in Fig. 35, mutative operational semantics accesses the memory M while denotational operational semantics locally operates on the body of Ref. Crucially, in denotational semantics, the reference update **updateRef** records the update $p \leftarrow y$ to the history H for each borrow path p of the mutable

$$\begin{array}{c}
\frac{\overline{y = \bar{n}} \in \hat{E} \quad m = \text{iop } \bar{n}}{x = \text{iop } \bar{y}, \hat{E}; \bar{M}; x \rightarrow x = m, \hat{E}; \bar{M}; x} \quad \frac{\overline{y = \bar{n}} \in \hat{E} \quad b = \text{asBool}(\text{irel } \bar{n})}{x = \text{irel } \bar{y}, \hat{E}; \bar{M}; x \rightarrow x = b, \hat{E}; \bar{M}; x} \quad \frac{y = \hat{v}, z = \hat{w} \in \hat{E}}{x = \text{par } y \ z, \hat{E}; \bar{M}; x \rightarrow x = (y, z), \hat{E}; \bar{M}; x} \\
\\
\frac{y = \hat{v} \in \hat{E}}{x = \text{consume } y, \hat{E}; \bar{M}; x \rightarrow x = (), \hat{E}; \bar{M}; x} \quad \frac{y = \hat{v} \in \hat{E}}{x = \text{move } y, \hat{E}; \bar{M}; x \rightarrow x = \text{Ur } y, \hat{E}; \bar{M}; x} \\
\\
\frac{f = \hat{v} \in \hat{E}}{x = \text{linearly } f, \hat{E}; \bar{M}; x \rightarrow li = \emptyset, y = f \ li, x = \text{linear } y, \hat{E}; \bar{M}; x} \quad \frac{y = \hat{v} \in \hat{E}}{x = \text{linear } y, \hat{E}; \bar{M}; x \rightarrow x = \hat{v}, \hat{E}; \bar{M}; x} \\
\\
\frac{y = \hat{v} \in \hat{E}}{x = \text{withLinearly } y, \hat{E}; \bar{M}; x \rightarrow li = \emptyset, x = (li, y), \hat{E}; \bar{M}; x}
\end{array}$$

(a) For mutative operational semantics.

$$\begin{array}{c}
\frac{\overline{y = \bar{n}} \in \hat{E} \quad m = \text{iop } \bar{n}}{x = \text{iop } \bar{y}, \hat{E}; x \rightarrow x = m, \hat{E}; x} \quad \frac{\overline{y = \bar{n}} \in \hat{E} \quad b = \text{asBool}(\text{irel } \bar{n})}{x = \text{irel } \bar{y}, \hat{E}; x \rightarrow x = b, \hat{E}; x} \quad \frac{y = \hat{v}, z = \hat{w} \in \hat{E}}{x = \text{par } y \ z, \hat{E}; x \rightarrow x = (y, z), \hat{E}; x} \\
\\
\frac{y = \hat{v} \in \hat{E}}{x = \text{consume } y, \hat{E}; x \rightarrow x = (), \hat{E}; x} \quad \frac{y = \hat{v} \in \hat{E}}{x = \text{move } y, \hat{E}; x \rightarrow x = \text{Ur } y, \hat{E}; x} \\
\\
\frac{f = \hat{v} \in \hat{E}}{x = \text{linearly } f, \hat{E}; x \rightarrow li = \emptyset, y = f \ li, x = \text{linear } y, \hat{E}; x} \quad \frac{y = \hat{v} \in \hat{E}}{x = \text{linear } y, \hat{E}; x \rightarrow x = \hat{v}, \hat{E}; x} \quad \frac{y = \hat{v} \in \hat{E}}{x = \text{withLinearly } y, \hat{E}; x \rightarrow li = \emptyset, x = (li, y), \hat{E}; x}
\end{array}$$

(b) For denotational operational semantics.

Fig. 34. Reduction rules for basic operators.

$$\begin{array}{c}
\frac{li = \emptyset, y = \hat{v} \in \hat{E}}{x = \text{newRef } li \ y, \hat{E}; \bar{M}; x \rightarrow x = \text{Ref } \ell, \hat{E}; \ell \mapsto y, \bar{M}; x} \quad \frac{li = \emptyset, y = \hat{v} \in \hat{E}}{x = \text{newRef } li \ y, \hat{E}; x \rightarrow x = \text{Ref } y, \hat{E}; x} \\
\\
\frac{y = \text{Ref } \ell \in \hat{E}}{x = \text{freeRef } y, \hat{E}; \ell \mapsto z, \bar{M}; x \rightarrow x = z, \hat{E}; \bar{M}; x} \quad \frac{y = \text{Ref } z \in \hat{E}}{x = \text{freeRef } y, \hat{E}; x \rightarrow x = z, \hat{E}; x}
\end{array}$$

(a) For mutative semantics.

(b) For denotational semantics.

Fig. 35. Basic reference-related reduction rules.

borrower. Also, the dereference `deref` distributes the borrower constructor, modifying the borrow path, just like in `DRED-CASE-BOR`.

B Association System

We present the omitted details of our association system introduced in §5.4.

B.1 Overview

Syntax. The syntax for the association system is shown in Fig. 39.

A lifetime path $\rho = \bar{u}.i$ represents a part of an atomic lifetime `al` $\bar{u}$. We introduce extended types $\hat{T}$, which include `DoneP T` to represent the result of `exeBO`/`exeBOH` and the skeleton type `Skel T` to represent a borrower whose lender has already reclaimed. The resource context $\hat{\Gamma}$ extends the typing context Γ with more information about the dynamic ownership.³⁵ A variable-type binding $x ::_{\pi}^P \hat{T}$ in a resource context is marked with the set of borrow paths P that the variable may be

³⁵ The order of items in a resource context $\hat{\Gamma}$ is ignored.

$$\begin{array}{c}
\frac{li = \emptyset, f = \hat{v} \in \hat{E}}{x = \text{newLifetime } li \ f, \hat{E}; M; x \rightarrow \text{now} = \emptyset, x = f \ \text{now}, \hat{E}; M; x} \\
\frac{li = \emptyset, y = \hat{v} \in \hat{E}}{x = \text{borrow } li \ y, \hat{E}; M; x \rightarrow y_m = \hat{v}, y_l = \hat{v}, x = (y_m, y_l), \hat{E}; M; x} \\
\frac{y = \hat{v} \in \hat{E}}{x = \text{copy } y, \hat{E}; M; x \rightarrow x = \hat{v}, \hat{E}; M; x} \\
\frac{y = \hat{v} \in \hat{E}}{x = \text{joinMut } y, \hat{E}; M; x \rightarrow x = \hat{v}, \hat{E}; M; x} \\
\frac{end = \emptyset, y = \hat{v} \in \hat{E}}{x = \text{reclaim } y \ end, \hat{E}; M; x \rightarrow x = y, \hat{E}; M; x} \\
\frac{now = \emptyset \in \hat{E}}{x = \text{endLifetime } now, \hat{E}; M; x \rightarrow x = \text{Ur } now, \hat{E}; M; x} \\
\frac{y = \hat{v} \in \hat{E}}{x = \text{share } y, \hat{E}; M; x \rightarrow x = \text{Ur } y, \hat{E}; M; x}
\end{array}$$

(a) For mutative operational semantics.

$$\begin{array}{c}
\frac{li = \emptyset, f = \hat{v} \in \hat{E}}{x = \text{newLifetime } li \ f, \hat{E}; x \rightarrow \text{now} = \emptyset, x = f \ \text{now}, \hat{E}; x} \\
\frac{li = \emptyset, y = \hat{v} \in \hat{E}}{x = \text{borrow } li \ y, \hat{E}; x \rightarrow y_m = \text{Mut}^{\delta} \hat{v}, y_l = \text{Lend}^{\delta} y, x = (y_m, y_l), \delta, \hat{E}; x} \\
\frac{y = \text{Share } \hat{v} \in \hat{E}}{x = \text{copy } y, \hat{E}; x \rightarrow x = \hat{v}, \hat{E}; x} \\
\frac{y = \hat{B}(\text{Mut}^{\hat{q}} \hat{v}) \in \hat{E} \quad \hat{B}' = \begin{cases} \text{Mut}^{\hat{p}, \hat{q}} \hat{B} = \text{Mut}^{\hat{p}} \\ \text{Share } \hat{B} = \text{Share} \end{cases}}{x = \text{joinMut } y, \hat{E}; x \rightarrow x = \hat{B}' \hat{v}, \hat{E}; x} \text{DRED-JOIN-BOR} \\
\frac{end = \emptyset_H, y = \text{Lend}^{\delta} z \in \hat{E} \quad \text{restore}_H \delta \hat{E} \stackrel{z}{\rightsquigarrow} z' \hat{E}'}{x = \text{reclaim } y \ end, \hat{E}; x \rightarrow x = z', \hat{E}'; x} \text{DRED-RECLAIM}
\end{array}$$

(b) For denotational operational semantics.

$$\begin{array}{c}
\frac{\forall \bar{i}. \bar{p}. \bar{i} \notin \text{dom } H}{\text{restore}_H \bar{p} \hat{E} \stackrel{x}{\rightsquigarrow} x' \hat{E}} \quad \frac{\bar{p} \leftarrow y \in H \quad x = \text{Ref } z \in \hat{E} \quad \text{restore}_H \bar{p}.0 \hat{E} \stackrel{y}{\rightsquigarrow} y' \hat{E}'}{\text{restore}_H \bar{p} \hat{E} \stackrel{x}{\rightsquigarrow} x' \hat{E} \quad x' = \text{Ref } y', \hat{E}'} \\
\frac{\bar{p} \notin \text{dom } H \quad \bar{p}. \bar{j} \in \text{dom } H \quad x = \hat{B}(\text{Ref } y) \in \hat{E} \quad \text{restore}_H \bar{p}.0 \hat{E} \stackrel{y}{\rightsquigarrow} y' \hat{E}'}{\text{restore}_H \bar{p} \hat{E} \stackrel{x}{\rightsquigarrow} x' \hat{E} \quad x' = \hat{B}(\text{Ref } y), \hat{E}'} \\
\frac{\bar{p} \notin \text{dom } H \quad \bar{p}. \bar{j} \in \text{dom } H \quad x = \hat{B}(C \bar{y}^n) \in \hat{E} \quad \hat{E}_0 = \hat{E} \quad \forall i. \text{restore}_H \bar{p}.i \hat{E}_i \stackrel{y_i}{\rightsquigarrow} y'_i \hat{E}_{i+1}}{\text{restore}_H \bar{p} \hat{E} \stackrel{x}{\rightsquigarrow} x' \hat{E} \quad x' = \hat{B}(C \bar{y}'), \hat{E}_n}
\end{array}$$

(c) The restoration-by-history predicate $\text{restore}_H \bar{p} \hat{E} \stackrel{x}{\rightsquigarrow} y' \hat{E}'$.

Fig. 36. Borrow-related reduction rules.

able to update (we omit the superscript P if it is empty). This information is vital to justify **parBO**, ensuring that the borrow paths updated by the two monad arguments are mutually disjoint. A resource context also contains borrow ids δ , variable-term bindings $x = \hat{t} \propto \hat{t}$, $x = \hat{v}$, and points-to tokens $\ell \mapsto x$ from the environments and memory. Importantly, bindings $x = \hat{v} \propto \hat{v}$, $x = \hat{v}$ over values are shareable. A captured points-to token $\ell \mapsto x$ represents a usual points-to token captured inside linear (please see **ASSOC-LINEAR** shown later).

Interestingly, a resource context can even contain polymorphic resource contexts $\forall x. \hat{\Gamma}$ to reason about *first-class polymorphism* of Haskell. For example, we can create a polymorphic reference $\forall X. \text{Ref } [X]$ to a list of any type from the nil list $[\] :: \forall X. [X]$, and then borrow it to create a polymorphic pair $\forall a, X. (\text{Mut}^a(\text{Ref } [X]), \text{Lend}^a(\text{Ref } [X]))$ of a borrower and a lender. Because the variables a, X are instantiated dynamically, we have to keep the polymorphism over a, X in creating the ghost state for the borrow.

$$\begin{array}{c}
\frac{\text{now} = \emptyset, \text{bo} = \hat{v} \in \hat{E}}{x = \text{execBO now bo, } \hat{E}; M; x \rightarrow \text{res} = \text{exeBO bo, } x = \text{execBO}^\circ \text{ res, } \hat{E}; M; x} \\
\frac{\text{bo} = \text{pure } y \in \hat{E}}{x = \text{exeBO bo, } \hat{E}; M; x \rightarrow x = \text{Done } y, \hat{E}; M; x} \\
\frac{\text{res} = \text{Done}_H y \in \hat{E}}{x = \text{res} \gg^\circ \text{ ko}; \hat{E}; M; x \rightarrow \text{bo} = \text{ko } y, x = \text{exeBO bo, } \hat{E}; M; x} \\
\frac{\text{now} = \emptyset, \text{bo}' = \hat{v} \in \hat{E}}{x = \text{sexecBO}^+ \text{ now bo}', \hat{E}; M; x \rightarrow \text{res} = \text{exeBO bo}', x = \text{sexecBO}^\circ \text{ res, } \hat{E}; M; x} \\
\frac{\text{bo} = \text{parBO bo}_0 \text{ bo}_1 \in \hat{E}}{x = \text{exeBO bo, } \hat{E}; M; x \rightarrow \text{res}_0 = \text{exeBO bo}_0, \text{res}_1 = \text{exeBO bo}_1, x = \text{parBO}^\circ \text{ res}_0 \text{ res}_1, \hat{E}; M; x} \\
\frac{\text{res} = \text{Done } y \in \hat{E}}{x = \text{execBO}^\circ \text{ res, } \hat{E}; M; x \rightarrow \text{now} = \emptyset, x = (\text{now}, y), \hat{E}; M; x} \\
\frac{\text{bo} = \text{bo}' \gg \text{ko} \in \hat{E}}{x = \text{exeBO bo, } \hat{E}; M; x \rightarrow \text{res} = \text{exeBO bo}', x = \text{res} \gg^\circ \text{ ko, } \hat{E}; M; x} \\
\frac{\text{bo} = \text{sexecBO now bo}' \in \hat{E}}{x = \text{exeBO bo, } \hat{E}; M; x \rightarrow x = \text{sexecBO}^+ \text{ now bo}', \hat{E}; M; x} \\
\frac{\text{res} = \text{Done } y \in \hat{E}}{x = \text{sexecBO}^\circ \text{ res, } \hat{E}; M; x \rightarrow x = \text{Done res, } \hat{E}; M; x} \\
\frac{\text{res}_0 = \text{Done } y, \text{res}_1 = \text{Done } z \in \hat{E}}{x = \text{parBO}^\circ \text{ res}_0 \text{ res}_1; \hat{E}; M; x \rightarrow \text{pr} = (y, z), x = \text{Done pr, } \hat{E}; M; x}
\end{array}$$

(a) For mutative operational semantics.

$$\begin{array}{c}
\frac{\text{now} = \emptyset_H, \text{bo} = \hat{v} \in \hat{E}}{x = \text{execBO now bo, } \hat{E}; x \rightarrow \text{res} = \text{exeBO}_H \text{ bo, } x = \text{execBO}^\circ \text{ res, } \hat{E}; x} \\
\frac{\text{bo} = \text{pure } y \in \hat{E}}{x = \text{exeBO}_H \text{ bo, } \hat{E}; x \rightarrow x = \text{Done}_H y, \hat{E}; x} \\
\frac{\text{res} = \text{Done}_H y \in \hat{E}}{x = \text{res} \gg^\circ \text{ ko}; \hat{E}; x \rightarrow \text{bo} = \text{ko } y, x = \text{exeBO}_H \text{ bo, } \hat{E}; x} \\
\frac{\text{now} = \emptyset_{H'}, \text{bo}' = \hat{v} \in \hat{E}}{x = \text{sexecBO}_H^+ \text{ now bo}', \hat{E}; x \rightarrow \text{res} = \text{exeBO}_\emptyset \text{ bo}', x' = \text{sexecBO}_{H;H'}^\circ \text{ res, } \hat{E}; x} \\
\frac{\text{bo} = \text{parBO bo}_0 \text{ bo}_1 \in \hat{E}}{x = \text{exeBO}_H \text{ bo, } \hat{E}; x \rightarrow \text{res}_0 = \text{exeBO}_\emptyset \text{ bo}_0, \text{res}_1 = \text{exeBO}_\emptyset \text{ bo}_1, x = \text{parBO}_H^\circ \text{ res}_0 \text{ res}_1, \hat{E}; x} \\
\frac{\text{res} = \text{Done}_H y \in \hat{E}}{x = \text{execBO}^\circ \text{ res, } \hat{E}; x \rightarrow \text{now} = \emptyset_H, x = (\text{now}, y), \hat{E}; x} \\
\frac{\text{bo} = \text{bo}' \gg \text{ko} \in \hat{E}}{x = \text{exeBO}_H \text{ bo, } \hat{E}; x \rightarrow \text{res} = \text{exeBO}_H \text{ bo}', x = \text{res} \gg^\circ \text{ ko, } \hat{E}; x} \\
\frac{\text{bo} = \text{sexecBO now bo}' \in \hat{E}}{x = \text{exeBO}_H \text{ bo, } \hat{E}; x \rightarrow x = \text{sexecBO}_H^+ \text{ now bo}', \hat{E}; x} \\
\frac{\text{res} = \text{Done}_{H_0} y \in \hat{E}}{x = \text{sexecBO}_{H;H'}^\circ \text{ res, } \hat{E}; x \rightarrow \text{now} = \emptyset_{H' / H_0}, \text{pr} = (\text{now}, y), x = \text{Done}_{H / H_0} \text{ pr, } \hat{E}; x} \\
\frac{\text{res}_0 = \text{Done}_{H_0} y, \text{res}_1 = \text{Done}_{H_1} z \in \hat{E}}{x = \text{parBO}_H^\circ \text{ res}_0 \text{ res}_1; \hat{E}; x \rightarrow \text{pr} = (y, z), x = \text{Done}_{H / (H_0 \cdot H_1)} \text{ pr, } \hat{E}; x}
\end{array}$$

(b) For denotational operational semantics.

Fig. 37. Basic monadic reduction rules.

A resource context can contain ghost resources *gho* to represent various auxiliary information. The meanings of ghost resources are as follows. A linearity witness **LINEAR** is used for Linearly. A live lifetime token $\text{NOW}_{H;P}^{\iota; \bar{i}; n}$ owns the part $\bar{i}$ of the lifetime **al** ι , asserting that it is alive with the partial history H , has n children tokens, and has the right to perform updates of borrow paths in P . A dead lifetime token END_H^ι asserts that the lifetime **al** ι has ended with the history H . A borrow lifetime token $\text{LIFETIME}_\alpha^\delta$ asserts that the lifetime of the borrow of the id δ is α . A borrow path token $\text{BORROW}_{x;T;\bar{q}}^{\bar{p}}$ asserts that the borrow of the borrow path sequence $\bar{p}$ owns an object of

$$\begin{array}{c}
\frac{bo = \text{deref } ref \in \hat{E}}{x = \text{exeBO } bo, \hat{E}; M; x \rightarrow x = \text{deref}^\circ ref, \hat{E}; M; x} \quad \frac{ref = \text{Ref } \ell \in \hat{E} \quad \ell \mapsto y \in M}{x = \text{deref}^\circ ref, \hat{E}; M; x \rightarrow y' = y, x = \text{Done } y', \hat{E}; M; x} \\
\\
\frac{bo = \text{updateRef } ko \text{ ref} \in \hat{E}}{x = \text{exeBO } bo, \hat{E}; M; x \rightarrow x = \text{updateRef}^+ ko \text{ ref}, \hat{E}; M; x} \quad \frac{ko = \hat{v}, ref = \text{Ref } \ell \in \hat{E} \quad \ell \mapsto y \in M}{x = \text{updateRef}^+ ko \text{ ref}, \hat{E}; M; x \rightarrow bo = ko y, res = \text{execBO } bo, x = \text{updateRef}_\ell^+ res, \hat{E}; M; x} \\
\\
\frac{res = \text{Done } pr \in \hat{E}}{x = \text{updateRef}_\ell^+ res, \hat{E}; M; x \rightarrow x = \text{updateRef}_\ell^+ pr, \hat{E}; M; x} \quad \frac{pr = (z, y) \in \hat{E}}{x = \text{updateRef}_\ell^+ pr, \hat{E}; M; x \rightarrow ref = \text{Ref } \ell, pr' = (z, ref), x = \text{Done } pr', \hat{E}; M\{\ell \leftarrow y\}; x}
\end{array}$$

(a) For mutative operational semantics.

$$\begin{array}{c}
\frac{bo = \text{deref } ref \in \dot{E}}{x = \text{exeBO}_H bo, \dot{E}; x \rightarrow x = \text{deref}_H^\circ ref, \dot{E}; x} \quad \frac{ref = \dot{B}(\text{Ref } y) \in \dot{E} \quad \dot{B}' = \begin{cases} \text{Mut}^{\bar{p}, \bar{0}} & \dot{B} = \text{Mut}^{\bar{p}} \\ \text{Share} & \dot{B} = \text{Share} \end{cases}}{x = \text{deref}_H^\circ ref, \dot{E}; x \rightarrow y' = \dot{B}' y, x = \text{Done}_H y', \dot{E}; x} \\
\\
\frac{bo = \text{updateRef } ko \text{ ref} \in \dot{E}}{x = \text{exeBO}_H bo, \dot{E}; x \rightarrow x = \text{updateRef}_H^+ ko \text{ ref}, \dot{E}; x} \quad \frac{ko = \hat{v}, ref = \text{Mut}^{\bar{p}}(\text{Ref } y) \in \dot{E}}{x = \text{updateRef}_H^+ ko \text{ ref}, \dot{E}; x \rightarrow bo = ko y, res = \text{exeBO}_H bo, x = \text{updateRef}_{\bar{p}}^+ res, \dot{E}; x} \\
\\
\frac{res = \text{Done}_H pr \in \dot{E}}{x = \text{updateRef}_{\bar{p}}^+ res, \dot{E}; x \rightarrow x = \text{updateRef}_{\bar{p}, H}^+ pr, \dot{E}; x} \quad \frac{pr = (z, y) \in \dot{E}}{x = \text{updateRef}_{\bar{p}, H}^+ pr, \dot{E}; x \rightarrow ref = \text{Mut}^{\bar{p}}(\text{Ref } y), pr' = (z, ref), x = \text{Done}_{H/\bar{p} \leftarrow y} pr', \dot{E}; x}
\end{array}$$

(b) For denotational operational semantics.

Fig. 38. Reference-related monadic reduction rules.

$$\begin{array}{ll}
\text{Lifetime path } \rho ::= \iota \mid \rho.i & \text{Extended type } \mathring{T} ::= T \mid \text{Done}^{\bar{p}} T \mid \text{Skel } T \\
\text{Ghost resource } gho ::= \text{LINEAR} \mid \text{NOW}_{H;P}^{\rho;n} \mid \text{END}_H^L \mid \text{LIFETIME}_\alpha^\delta \mid \text{BORROW}_{x;T;\bar{q}}^{\bar{p}} \\
& \mid \text{MUT}^{\bar{p}} \mid \text{SHARE}^{\bar{p}} \mid \text{LEND}_{x;T}^\delta \mid \text{DEPO}^{\bar{p}} \mid \text{GDEPO}_{x;T;H}^\delta \\
& \mid \text{HOLE}^{\bar{p}} \mid \text{RELEND}_H^{\bar{p}} \mid \ell \xRightarrow{\bar{p}} x \mid \ell \xrightarrow{\bar{p}} x \mid \text{SKEL}_T^x \mid \text{VAR}_T^x \\
\text{Atomic resource } atom ::= x ::_{\pi}^P \mathring{T} \mid \delta \mid x = \hat{t} \propto \hat{t} \mid x = \hat{v} \mid \ell \mapsto x \mid \ell \mapsto x \mid \forall x. \mathring{\Gamma} \mid gho \\
\text{Resource context } \mathring{\Gamma}, \mathring{\Delta} ::= \overline{atom}
\end{array}$$

Fig. 39. Syntax for the association system.

a variable x typed T . When $\bar{q}$ is non-empty, it means that the object is also reborrowed under $\bar{q}$. When $\bar{q}$ is empty, we omit the last part ‘ $\bar{q}$ ’ of the subscript. We have tokens for a mutable borrower $\text{MUT}^{\bar{p}}$, a shared borrower $\text{SHARE}^{\bar{p}}$, and a lender $\text{LEND}_{x;T}^\delta$. We also have tokens for a (local) deposit $\text{DEPO}^{\bar{p}}$ and a global deposit $\text{GDEPO}_{x;T;H}^\delta$ for the borrowed contents. The hole token $\text{HOLE}^{\bar{p}}$ is used inside a global deposit to indicate a hole caused by a deposit $\text{DEPO}^{\bar{p}}$. The relender token $\text{RELEND}_H^{\bar{p}}$ asserts that the relender of $\bar{p}$ is under the updates of H . We also have a shared-borrowed points-to token $\ell \xRightarrow{\bar{p}} x$ and the evidence of the ownership $\ell \xrightarrow{\bar{p}} x$. Also, we have a shareable token SKEL_T^x for the skeleton type $\text{Skel } T$ and a variable token VAR_T^x providing skeletons with the witness that the variable x is typed T .

$$\begin{array}{c}
\frac{\overline{y = \hat{u} \propto \hat{u}}, \overline{y' = \hat{v} \propto \hat{v}}, \overline{y' = \hat{v}}, \overline{z = \hat{w}}, \overline{\delta}, \overline{M} \vdash \hat{t} \propto \hat{t} :: T}{x = \hat{t}, y = \hat{u}, y' = \hat{v}, \hat{E}; M; x \propto_T x = \hat{t}, y = \hat{u}, y' = \hat{v}, z = \hat{w}, \overline{\delta}, \hat{E}; x} \text{ CONFIG-ASSOC} \\
\\
\frac{\overline{\hat{\Delta} \preccurlyeq \hat{\Gamma}} \quad \overline{\hat{\Gamma} \vdash \hat{t} \propto \hat{t} :: \hat{T}}}{\overline{\hat{\Delta} \vdash \hat{t} \propto \hat{t} :: \hat{T}}} \text{ ASSOC-}\preccurlyeq \quad \frac{\overline{\hat{\Gamma} \vdash \hat{t} \propto \hat{t} :: T}}{(\forall x. \hat{\Gamma}) \vdash \hat{t} \propto \hat{t} :: \forall x. T} \text{ ASSOC-}\forall \\
\\
\frac{\forall i. \hat{\Gamma}_i \text{ real} \quad \forall i. \overline{x ::_{\omega} \hat{U}}, \hat{\Gamma}_i + \hat{\Delta} \vdash \hat{u}_i \propto \hat{u}_i :: \hat{U}_i \quad \overline{x ::_{\omega} \hat{U}}, \hat{\Delta}' \vdash \hat{t} \propto \hat{t} :: \hat{T}}{\overline{x = \hat{u} \propto \hat{u} + \sum_i \hat{\Gamma}_i + \omega \hat{\Delta} + \hat{\Delta}' \vdash \hat{t} \propto \hat{t} :: \hat{T}}} \text{ ASSOC-VAR-}\omega \\
\\
\frac{\overline{\hat{\Gamma} \text{ valid}} \quad \overline{\hat{\Gamma}' \text{ real}} \quad \overline{\hat{\Gamma} + \hat{\Gamma}' \vdash \hat{u} \propto \hat{u} :: \hat{U}} \quad \overline{x ::_{\pi}^{\text{BorPaths}} \hat{\Gamma}} \quad \overline{\hat{U}, \hat{\Delta} \vdash \hat{t} \propto \hat{t} :: \hat{T}}}{\overline{x = \hat{u} \propto \hat{u} + \hat{\Gamma}' + \pi \hat{\Gamma} + \hat{\Delta} \vdash \hat{t} \propto \hat{t} :: \hat{T}}} \text{ ASSOC-VAR} \\
\\
\frac{\overline{\hat{\Gamma} \text{ ghost, gvalid}} \quad \overline{\hat{\Gamma}, \ell \mapsto x, \hat{\Delta} \vdash x \propto x :: \text{Ur } T}}{\overline{\ell \mapsto x, \hat{\Delta} \vdash \text{linear } x \propto \text{linear } x :: \text{Ur } T}} \text{ ASSOC-LINEAR} \quad \text{LINEAR} \vdash \emptyset \propto \emptyset :: \text{Linearly} \\
\\
\ell \mapsto x, x ::_1^P T \vdash \text{Ref } \ell \propto \text{Ref } x :: \text{Ref } T \quad \text{NOW}_{H; \text{BorPath}}^{\iota; 0} \vdash \emptyset \propto \emptyset_H :: \text{Now}^{\text{al } \iota} \quad \frac{\alpha \leq \text{al } \iota}{\text{END}_H^{\iota} \vdash \emptyset \propto \emptyset_H :: \text{End}^{\alpha}} \\
\\
\frac{\overline{\forall i. P'_i \subseteq P_i} \quad \overline{\text{NOW}_{H; P}^{\rho; 0}, \text{bo} ::_1^{P'} \text{BO} \wedge \text{al}[P] T \vdash} \quad \overline{\text{NOW}_{H; P}^{\rho; 0}, x ::_1^{P'} T \vdash \text{Done } x \propto \text{Done}_H x :: \text{Done}^{\bar{P}} T}}{\overline{\text{exeBO } \text{bo} \propto \text{exeBO}_H \text{bo} :: \text{Done}^{\bar{P}} T}} \\
\\
\frac{\overline{\beta \leq \bigwedge_i \alpha_i} \quad \overline{T \preccurlyeq U} \quad \overline{U \preccurlyeq T}}{\overline{\text{MUT}^{\bar{P}}, \text{LIFETIME}_{\alpha}^{[P]}, \text{BORROW}_{x; T}^{\bar{P}} \vdash x \propto \text{Mut}^{\bar{P}} x :: \text{Mut}^{\beta} U}} \quad \frac{\alpha \leq \text{al } \iota \quad \overline{\hat{\Gamma} \vdash \hat{t} \propto \hat{t} :: \text{Skel } T}}{\overline{\text{END}_H^{\iota} + \hat{\Gamma} \vdash \hat{t} \propto \text{Mut}^{\bar{P}} \hat{t} :: \text{Mut}^{\alpha} T}} \\
\\
\frac{\overline{\beta \leq \bigwedge_i \alpha_i} \quad \overline{T \preccurlyeq U}}{\overline{\text{SHARE}^{\bar{P}}, \text{LIFETIME}_{\alpha}^{[P]}, \text{BORROW}_{x; T}^{\bar{P}} \vdash x \propto \text{Share } x :: \text{Share}^{\beta} U}} \quad \frac{\alpha \leq \text{al } \iota \quad \overline{\hat{\Gamma} \vdash \hat{t} \propto \hat{t} :: \text{Skel } T}}{\overline{\text{END}_H^{\iota} + \hat{\Gamma} \vdash \hat{t} \propto \text{Share } \hat{t} :: \text{Share}^{\alpha} T}} \\
\\
\frac{\overline{\alpha \leq \beta} \quad \overline{T \preccurlyeq U}}{\overline{\text{LEND}_{x; T}^{\delta}, \delta, \text{LIFETIME}_{\alpha}^{\delta}, x = \hat{v} \propto \hat{v} \vdash \hat{v} \propto \text{Lend}^{\delta} x :: \text{Lend}^{\beta} U}} \\
\\
\frac{\overline{\hat{\Gamma} \vdash x \propto x :: T} \quad \overline{\hat{\Delta} \vdash \hat{t} \propto \hat{t} :: \hat{U}}}{\overline{\text{DEPO}^{\bar{P}}, \text{BORROW}_{x; T}^{\bar{P}} + \hat{\Gamma} + \hat{\Delta} \vdash \hat{t} \propto \hat{t} :: \hat{U}}} \quad \frac{\overline{\hat{\Gamma} \vdash_H^{\delta} x :: T} \quad \overline{\hat{\Delta} \vdash \hat{t} \propto \hat{t} :: \hat{U}}}{\overline{\text{GDEPO}_{x; T; H}^{\delta}, \hat{\Gamma} + \hat{\Delta} \vdash \hat{t} \propto \hat{t} :: \hat{U}}} \\
\\
\frac{\overline{\hat{\Gamma} \vdash_H^{\bar{P}} \hat{t} \propto \hat{t} :: \hat{T}}}{\overline{x = \hat{t} \propto \hat{t} + \hat{\Gamma} \vdash_H^{\bar{P}} x :: \hat{T}}} \quad \frac{\overline{\hat{\Gamma} \vdash_H^{\bar{P}} \hat{v} \rightsquigarrow \hat{w}}}{\overline{\text{HOLE}^{\bar{P}}, \text{BORROW}_{x; T; \hat{q}}^{\bar{P}} x = \hat{v} \propto \hat{w}, \hat{\Gamma} \vdash_H^{\bar{P}} \hat{v} \propto \hat{v} :: T}} \quad \frac{\overline{\hat{\Gamma} \vdash \hat{t} \propto \hat{t} :: \hat{T}}}{\overline{\hat{\Gamma} \vdash_H^{\bar{P}} \hat{t} \propto \hat{t} :: \hat{T}}} \\
\\
\frac{\overline{\hat{\Gamma} \vdash_H^{\bar{P}, 0} y :: T}}{\overline{\ell \mapsto y, \ell \xrightarrow{\bar{P}} y, \hat{\Gamma} \vdash_{\bar{P} \leftarrow y, H}^{\bar{P}} \text{Ref } \ell \propto \text{Ref } x :: \text{Ref } T}} \quad \frac{\overline{\forall i. p_i \notin \text{dom } H} \quad \overline{H \neq \emptyset} \quad \overline{\hat{\Gamma} \vdash_H^{\bar{P}, 0} x :: T}}{\overline{\ell \mapsto x, \ell \xrightarrow{\bar{P}} x, \hat{\Gamma} \vdash_H^{\bar{P}} \text{Ref } \ell \propto \text{Ref } x :: \text{Ref } T}}
\end{array}$$

Fig. 40. Selected association rules.

Association rules. Selected rules for the association judgment are shown in Fig. 40.³⁶ Appendix B.2 presents the omitted rules and formal definitions.

The association relation over mutative and denotational configurations is $\hat{G} \propto_T \hat{G}$. Its only rule is **CONFIG-ASSOC**, which puts into the resource context the variable-term bindings, the borrow ids of the environments, and the points-to tokens of the memory, discarding some old variables.

The association judgment has the form $\hat{\Gamma} \vdash \hat{t} \propto \hat{t} :: \hat{T}$. The association judgment is closed under resource context inclusion (**ASSOC- $\preccurlyeq$**), defined naturally similarly to the typing context inclusion

³⁶ We define $\lfloor \rho \rfloor$ by $\lfloor \cdot, i \rfloor \triangleq \iota$ and $\lfloor p \rfloor$ by $\lfloor \delta, i \rfloor \triangleq \delta$.

Fig. 26. For polymorphic types, we use polymorphic resource contexts (**ASSOC-V**). For variable introduction, we use the condition $\dot{\Gamma}$ real, meaning that every item in $\dot{\Gamma}$ is of the form $x = \hat{t} \propto \hat{t}$ or $\ell \mapsto x$.

Unlike the unrestricted case (**ASSOC-VAR- ω**), introduction of a possibly linear variable (**ASSOC-VAR**) is tricky, because the set of accessible borrow paths should be determined from the resource context $\dot{\Gamma}$, where we define $\text{BorPaths } \dot{\Gamma}$ as the set of the subpaths of the borrow path of each mutable borrower owned by $\dot{\Gamma}$. For this purpose, we introduce the *validity* side condition $\dot{\Gamma}$ valid. For example, it ensures that $\dot{\Gamma}$ is free of a linear variable-type binding $x ::_1^P T$ and of overlapping live lifetime tokens. Also, to handle *nested borrowing* properly (which is much trickier than it might look), the validity ensures that $\dot{\Gamma}$ owns all the mutably borrowed objects accessible by $\dot{\Gamma}$.

The key is the rule **ASSOC-LINEAR** for linear, an intermediate state of linearly. Here, we can capture some fragment M of the memory and an arbitrary ghost resource context $\dot{\Gamma}$,³⁷ under the *global validity* side condition $\dot{\Gamma}$ gvalid, stronger than the usual validity valid. For example, the global validity ensures that $\dot{\Gamma}$ contains a borrowed object for each lender in $\dot{\Gamma}$. The association rule for linear itself (shown next) is straightforward.

A linearity token **Linearly** captures **LINEAR**, a reference **Ref** T captures a points-to token $\ell \mapsto x$ and a body, and live and dead lifetime tokens **Now** $^\alpha$, **End** $^\alpha$ capture **now** and **end** of the observed history H . The type **Done** $^\beta T$ also captures **now** of the lifetime paths $\bar{\rho}$. A mutable borrower is usually obtained from the token **MUT** $^{\bar{\rho}}$ accompanied with some shared information. A shared borrower is similarly obtained using **SHARE** $^{\bar{\rho}}$. Also, borrowers whose lifetime has ended can be obtained through a skeleton **Skel** T . A lender is obtained simply from the token **LEND** $_{x;T}^\delta$.

The content of a deposit **DEPO** $^{\bar{\rho}}$, **GDEPO** $_{x;T,H}^\delta$ is owned by some variable. The tricky point is which variable should own each deposit. We carefully design the validity condition (**ASSOC-VAR**) so that the borrower or the lender with a sufficient lifetime token should own the deposit, so that nested borrowers work properly in the presence of **parBO**. For deposits, we introduce an auxiliary judgment $\dot{\Gamma} \vdash_{\dot{H}}^{\bar{\rho}} \hat{t} \propto \hat{t} :: \dot{T}$, asserting that the object $\hat{t} \propto \hat{t}$ deposited at $\bar{\rho}$ becomes an object of type $\dot{T}$ after the updates of H . Its rules are defined using an auxiliary judgment $\dot{\Gamma} \vdash_{\dot{H}}^{\bar{\rho}} \hat{t} \rightsquigarrow \hat{u}$, meaning that a term $\hat{t}$ at $\bar{\rho}$ turns into $\hat{u}$ after the updates of H .

B.2 Details

Here we present more details of the association system.

Basics. The subtyping over extended types $\dot{T} \preceq \dot{T}'$ extends the usual subtyping $T \preceq T$ by the following rules:

$$\frac{T \preceq U}{\text{Done}^\alpha T \preceq \text{Done}^\alpha U} \quad \frac{T \preceq U}{\text{Skel } T \preceq \text{Skel } U}$$

We say an atomic resource *atom* is shareable and write *atom* shareable if *atom* is of the form $x ::_{\omega}^P \dot{T}$, $x = \hat{v} \propto \hat{v}$, $x = \hat{v}$, δ , **END** $_{\dot{H}}^{\bar{\rho}}$, **LIFETIME** $_{\alpha}^\delta$, **BORROW** $_{x;T;\bar{q}}^{\bar{\rho}}$, **SHARE** $^{\bar{\rho}}$, $\ell \xrightarrow{\bar{\rho}} x$, or **SKEL** $_T^x$.

The resource context inclusion $\dot{\Gamma} \preceq \dot{\Delta}$ is inductively defined by the following rules:

$$\begin{array}{c} \dot{\Gamma} \preceq \dot{\Gamma} \quad \frac{\dot{\Gamma} \preceq \dot{\Gamma}' \quad \dot{\Gamma}' \preceq \dot{\Gamma}''}{\dot{\Gamma} \preceq \dot{\Gamma}''} \quad \frac{\mu \leq \pi \quad T \preceq U}{x ::_{\pi}^P T, \dot{\Gamma} \preceq x ::_{\mu}^P U, \dot{\Gamma}} \\[10pt] \frac{\text{atom shareable}}{\text{atom}, \dot{\Gamma} \preceq \dot{\Gamma}} \quad (\forall x. \dot{\Gamma}), \dot{\Delta} \preceq \dot{\Gamma}[a/x], \dot{\Delta} \quad \frac{a \text{ is fresh in } \dot{\Delta} \text{ and } \dot{\Delta}' \quad \dot{\Delta}' \preceq \dot{\Gamma}, \dot{\Delta}}{\dot{\Delta}' \preceq (\forall x. \dot{\Gamma}), \dot{\Delta}} \end{array}$$

³⁷ The condition $\dot{\Gamma}$ ghost means that $\dot{\Gamma}$ consists only of ghost resources *gho* after unfolding polymorphism.

The multiplication $\pi \dot{\Gamma}$ over a resource context is defined as follows:

$$\begin{aligned} 1 \dot{\Gamma} &\triangleq \dot{\Gamma} & \pi(x ::_{\mu} T, \dot{\Gamma}) &\triangleq x ::_{\pi \cdot \mu} T, \pi \dot{\Gamma} \\ \pi((\forall x. \dot{\Gamma}), \dot{\Delta}) &\triangleq (\forall x. \pi \dot{\Gamma}), \pi \dot{\Delta} & \frac{\text{atom shareable}}{\pi(\text{atom}, \dot{\Gamma}) \triangleq \text{atom}, \pi \dot{\Gamma}} & \pi \emptyset \triangleq \emptyset \end{aligned}$$

The addition $\dot{\Gamma} + \dot{\Delta}$ of resource contexts is defined as follows:

$$\frac{\text{atom shareable}}{(\text{atom}, \dot{\Gamma}) + (\text{atom}, \dot{\Delta}) \triangleq \text{atom}, (\dot{\Gamma} + \dot{\Delta})} \quad \frac{\neg \text{atom shareable}}{(\text{atom}, \dot{\Gamma}) + \dot{\Delta} \triangleq \text{atom}, (\dot{\Gamma} + \dot{\Delta})} \quad \emptyset + \dot{\Delta} \triangleq \dot{\Delta}$$

The predicate $\dot{\Gamma}$ ghost is inductively defined by the following rules:

$$\frac{}{\overline{\text{gho}} \text{ ghost}} \quad \frac{\dot{\Gamma}, \dot{\Delta} \text{ ghost}}{(\forall x. \dot{\Gamma}), \dot{\Delta} \text{ ghost}}$$

On histories. The set of borrow paths $\text{BorPaths } \dot{\Gamma}$ for a resource context $\dot{\Gamma}$ is inductively defined by the following rules:

$$\frac{\text{MUT}^{\bar{p}} T \in \dot{\Gamma}}{p_i \cdot \bar{j} \in \text{BorPaths } \dot{\Gamma}} \quad \frac{p \in \text{BorPaths } (\dot{\Gamma}, \dot{\Delta})}{p \in \text{BorPaths } ((\forall x. \dot{\Gamma}), \dot{\Delta})}$$

The predicate $\text{Now}^{\rho} \dot{\Gamma} \rightsquigarrow H$, extracting the history H for the lifetime path ρ from the resource context $\dot{\Gamma}$ is inductively defined by the following rules:

$$\frac{\forall i < n. \text{Now}^{\rho \cdot i} \dot{\Gamma}_i \rightsquigarrow H'_i}{\text{Now}^{\rho} (\text{Now}_{H;P}^{\rho; n}, \dot{\Gamma}) \rightsquigarrow H / \prod_i H'_i} \quad \frac{\text{Now}^{\rho} \dot{\Gamma} \rightsquigarrow H \quad \text{atom is not of the form } \text{Now}_{H';P}^{[\rho] \cdot \bar{i}; n}}{\text{Now}^{\rho} (\text{atom}, \dot{\Gamma}) \rightsquigarrow H}$$

Importantly, the predicate is right-unique (i.e., unique in the history H). We also use the shorthand $\text{Now}^{\rho} \dot{\Gamma}$ for $\exists H. \text{Now}^{\rho} \dot{\Gamma}$.

The restriction $H|_p$ of a history H with respect to a borrow path p is defined as follows:

$$(p \cdot \bar{i} \leftarrow x, H)|_p \triangleq p \cdot \bar{i} \leftarrow x, (H|_p) \quad \frac{q \text{ is not of the form } p \cdot \bar{i}}{(q \leftarrow x, H)|_p \triangleq H|_p} \quad \emptyset|_p \triangleq \emptyset$$

Validity. The validity $\dot{\Gamma}$ valid of a resource context is defined as the negation of the invalidity $\dot{\Gamma}$ invalid, inductively defined by the following rules:

$$\begin{aligned} &\frac{x \text{ is fresh in } \dot{\Delta} \quad \dot{\Gamma}, \dot{\Delta} \text{ invalid}}{(\forall x. \dot{\Gamma}), \dot{\Delta} \text{ invalid}} & x ::_1^p T, \dot{\Gamma} \text{ invalid} \\ &\frac{\text{dom } H \not\subseteq P}{\text{Now}_{H;P}^{\rho; n}, \dot{\Gamma} \text{ invalid}} & \frac{P' \not\subseteq P}{\text{Now}_{H;P}^{\rho; n}, \text{Now}_{H';P'}^{\rho \cdot \bar{i}; m}, \dot{\Gamma} \text{ invalid}} & \frac{i \neq i' \quad P \cap P' \neq \emptyset}{\text{Now}_{H;P}^{\rho \cdot i; \bar{j}; n}, \text{Now}_{H';P'}^{\rho \cdot i' \cdot \bar{j}'; m}, \dot{\Gamma} \text{ invalid}} \\ &\text{Now}_{H;P}^{\rho; n}, \text{END}_{H'}^{[\rho]}, \dot{\Gamma} \text{ invalid} & \text{END}_{H'}^t, \text{END}_{H'}^t, \dot{\Gamma} \text{ invalid} \\ &\text{LIFETIME}_{\alpha}^{\delta} \text{LIFETIME}_{\beta}^{\delta} \dot{\Gamma} \text{ invalid} & \text{BORROW}_{x;T}^{\bar{p}}, \text{BORROW}_{y;U}^{\bar{p}}, \dot{\Gamma} \text{ invalid} \\ &\text{MUT}^{\bar{p}}, \text{MUT}^{\bar{p}}, \dot{\Gamma} \text{ invalid} & \text{MUT}^{\bar{p}}, \text{SHARE}^{\bar{p}}, \dot{\Gamma} \text{ invalid} & \text{LEND}_{x;T}^{\delta}, \text{LEND}_{y;U}^{\delta}, \dot{\Gamma} \text{ invalid} \\ &\text{DEPO}^{\bar{p}}, \text{DEPO}^{\bar{p}}, \dot{\Gamma} \text{ invalid} & \text{GDEPO}_{x;T;H}^{\delta}, \text{GDEPO}_{y;U;H'}^{\delta}, \dot{\Gamma} \text{ invalid} & \text{HOLE}^{\bar{p}}, \text{HOLE}^{\bar{p}}, \dot{\Gamma} \text{ invalid} \\ &\frac{\alpha \leq \text{al } \iota \quad \text{Now}^{\delta} \dot{\Gamma} \rightsquigarrow H' \quad H \neq H'|_{\delta}}{\text{GDEPO}_{x;T;H}^{\delta}, \text{LIFETIME}_{\alpha}^{\delta} \dot{\Gamma} \text{ invalid}} & \frac{\alpha \leq \text{al } \iota \quad H \neq H'|_{\delta}}{\text{GDEPO}_{x;T;H}^{\delta}, \text{LIFETIME}_{\alpha}^{\delta}, \text{END}_{H'}^t, \dot{\Gamma} \text{ invalid}} \end{aligned}$$

$$\begin{array}{c}
\frac{\alpha \leq \text{al } \iota \quad \text{Now}^\delta \dot{\Gamma} \rightsquigarrow H' \quad H|_{p_i} \neq H'|_{p_i}}{\text{RELEND}_{\dot{H}}^{\bar{p}}, \text{LIFETIME}_{\alpha}^{[p_i]}, \dot{\Gamma} \text{ invalid}} \quad \frac{\alpha \leq \text{al } \iota \quad H|_{p_i} \neq H'|_{p_i}}{\text{RELEND}_{\dot{H}}^{\bar{p}}, \text{LIFETIME}_{\alpha}^{[p_i]}, \text{END}_{H'}^{\iota}, \dot{\Gamma} \text{ invalid}} \\
\frac{\wedge_j \text{al } [p_j] \leq \wedge_i \alpha_i \quad \text{DEPO}^{\bar{p}} \notin \dot{\Gamma}}{\text{MUT}_{\dot{p}}, \text{BORROW}_{x;T}^{\bar{p}}, \text{LIFETIME}_{\alpha}^{[p]}, \text{NOW}_{H;P}^{\rho;n}, \dot{\Gamma} \text{ invalid}} \\
\frac{\alpha \leq \text{al } \iota \quad \forall H. \text{GDEPO}_{x;T;H}^{\delta} \notin \dot{\Gamma}}{\text{LEND}_{x;T}^{\delta}, \text{LIFETIME}_{\alpha}^{\delta}, \text{END}_{H'}^{\iota}, \dot{\Gamma} \text{ invalid}} \quad \frac{\alpha \leq \text{al } \iota \quad \text{Now}^\delta \dot{\Gamma} \quad \forall H. \text{GDEPO}_{x;T;H}^{\delta} \notin \dot{\Gamma}}{\text{LEND}_{x;T}^{\delta}, \text{LIFETIME}_{\alpha}^{\delta}, \dot{\Gamma} \text{ invalid}} \\
\frac{\alpha \leq \text{al } \iota \quad [p] = \delta \quad \text{DEPO}_{x;T;\bar{q}}^{\bar{p},p} \notin \dot{\Gamma}}{\text{LEND}_{x;T}^{\delta}, \text{LIFETIME}_{\alpha}^{\delta}, \text{END}_{H'}^{\iota}, \text{BORROW}_{x;T;\bar{q}}^{\bar{p},p}, \dot{\Gamma} \text{ invalid}} \quad \frac{\alpha \leq \text{al } \iota \quad \text{Now}^\delta \dot{\Gamma} \quad [p] = \delta \quad \text{DEPO}_{x;T;\bar{q}}^{\bar{p},p} \notin \dot{\Gamma}}{\text{LEND}_{x;T}^{\delta}, \text{LIFETIME}_{\alpha}^{\delta}, \text{BORROW}_{x;T;\bar{q}}^{\bar{p},p}, \dot{\Gamma} \text{ invalid}}
\end{array}$$

The global validity $\dot{\Gamma} \text{ gvalid}$ of a resource context is defined as the negation of the global invalidity $\dot{\Gamma} \text{ ginvalid}$, inductively defined by the following rules:

$$\begin{array}{c}
\frac{x \text{ is fresh in } \dot{\Delta} \quad \dot{\Gamma}, \dot{\Delta} \text{ ginvalid}}{(\forall x. \dot{\Gamma}), \dot{\Delta} \text{ ginvalid}} \quad \frac{\dot{\Gamma} \text{ invalid}}{\dot{\Gamma} \text{ ginvalid}} \quad \frac{\text{NOW}_{H';P}^{\rho;n} \in \dot{\Gamma} \quad \neg \text{Now}^{[p]} \dot{\Gamma}}{\dot{\Gamma} \text{ ginvalid}} \\
\frac{\text{DEPO}^{\bar{p}} \notin \dot{\Gamma}}{\text{BORROW}_{x;T;\bar{q}}^{\bar{p}}, \dot{\Gamma} \text{ ginvalid}} \quad \frac{\text{HOLE}^{\bar{p}} \notin \dot{\Gamma}}{\text{DEPO}^{\bar{p}}, \dot{\Gamma} \text{ ginvalid}} \quad \frac{\ell \xrightarrow{\bar{p}} x \notin \dot{\Gamma}}{\ell \xrightarrow{\bar{p}} x, \dot{\Gamma} \text{ ginvalid}} \\
\frac{\forall H. \text{GDEPO}_{x;T;H}^{\delta} \notin \dot{\Gamma}}{\text{LEND}_{x;T}^{\delta}, \dot{\Gamma} \text{ ginvalid}} \quad \frac{\text{LEND}_{x;T}^{\delta} \notin \dot{\Gamma}}{\text{GDEPO}_{x;T;H}^{\delta}, \dot{\Gamma} \text{ ginvalid}} \quad \frac{\forall x, T. \text{LEND}_{x;T}^{[p_i]} \notin \dot{\Gamma}}{\text{DEPO}^{\bar{p}}, \dot{\Gamma} \text{ ginvalid}} \quad \frac{\text{VAR}_T^x \notin \dot{\Gamma}}{\text{SKEL}_T^x, \dot{\Gamma} \text{ ginvalid}}
\end{array}$$

Association rules. The omitted rules for the association judgment $\dot{\Gamma} \vdash \dot{t} \propto \dot{t} :: \dot{T}$ are as follows. Here, we introduce an auxiliary judgment $a \propto :: \dot{T}; \dot{U}$ for operators and monad constructors a . Also, we write $\dot{\Gamma} \vdash \dot{\Delta}$ for $\dot{\Gamma}, \dot{\Delta}$ with the following side condition: for every item of the form $x ::_{\pi} \dot{T}$ in $\dot{\Gamma}$, x is fresh in $\dot{\Delta}$.

$$\begin{array}{c}
x ::_{\pi} \dot{T} \vdash x \propto x :: \dot{T} \\
\frac{\forall i. \dot{\Gamma}_i \vdash t_i \propto t_i :: T_i \quad \overline{y ::_i T_i} \vdash \dot{\Delta} \vdash u \propto u :: U}{\sum_i \dot{\Gamma}_i \vdash \dot{\Delta} \vdash \text{let } \overline{y = t} \text{ in } u \propto \text{let } \overline{y = t} \text{ in } u :: U} \quad \frac{\forall i. \overline{y ::_{\omega} T_i}, \dot{\Gamma} \vdash t_i \propto t_i :: T_i \quad \overline{y ::_{\omega} T_i} \vdash \dot{\Delta} \vdash u \propto u :: U}{\omega \dot{\Gamma} \vdash \dot{\Delta} \vdash \text{let } \overline{y = t} \text{ in } u \propto \text{let } \overline{y = t} \text{ in } u :: U} \\
\frac{x ::_{\pi} T, \dot{\Gamma} \vdash t \propto t :: U}{\dot{\Gamma} \vdash \lambda x \rightarrow t \propto \lambda x \rightarrow t :: T \rightarrow_{\pi} U} \quad \frac{\dot{\Gamma} \vdash t \propto t :: T \rightarrow_{\pi} U \quad \dot{\Delta} \vdash u \propto u :: T}{\dot{\Gamma} \vdash \pi \dot{\Delta} \vdash t u \propto t u :: U} \quad \frac{x \in \text{dom } \dot{\Gamma} \quad \dot{\Gamma} \vdash t \propto t :: T}{\dot{\Gamma} \vdash \text{seq } x t \propto \text{seq } x t :: T} \\
\vdash n :: \text{Int} \quad \frac{\forall i. \dot{\Gamma}_i \vdash t_i \propto t_i :: T_{j,i}[\overline{U/X}] \quad \text{data } F \bar{X} \text{ where } \overline{C :: \overline{T \rightarrow_{\pi} F \bar{X}}}}{\sum_i \pi_{j,i} \dot{\Gamma}_i \vdash C_j \bar{t} \propto C_j \bar{t} :: F \bar{U}} \\
\frac{\dot{\Gamma} \vdash t \propto t :: F \bar{U} \quad \forall j. x ::_{\pi_j} T_j[\overline{U/X}] \vdash \dot{\Delta} \vdash u_j :: U' \quad \text{data } F \bar{X} \text{ where } \overline{C :: \overline{T \rightarrow_{\pi} F \bar{X}}}}{\dot{\Gamma} \vdash \dot{\Delta} \vdash \text{case } t \text{ of } \{\overline{C \bar{x} \rightarrow u}\} \propto \text{case } t \text{ of } \{\overline{C \bar{x} \rightarrow u}\} :: U'} \\
\frac{\dot{\Gamma} \vdash t \propto t :: B^{\alpha}(F \bar{U}) \quad \forall j. x ::_{\pi_j} B^{\alpha}(T_j[\overline{U/X}]) \vdash \dot{\Delta} \vdash u_j :: U' \quad \text{data } F \bar{X} \text{ where } \overline{C :: \overline{T \rightarrow_{\pi} F \bar{X}}}}{\dot{\Gamma} \vdash \dot{\Delta} \vdash \text{case } t \text{ of } \{\overline{C \bar{x} \rightarrow u}\} \propto \text{case } t \text{ of } \{\overline{C \bar{x} \rightarrow u}\} :: U'} \\
\frac{a \propto :: \bar{T}; U \quad \forall i. \dot{\Gamma}_i \vdash t_i \propto t_i :: T_i}{\sum_i \dot{\Gamma}_i \vdash a \bar{t} \propto a \bar{t} :: U} \quad \text{iop } \propto :: \overline{\text{Int}}; \text{Int} \quad \text{irel } \propto :: \overline{\text{Int}}; \text{Bool} \quad \text{par } \propto :: T, U; (T, U) \\
\frac{T \text{ is Linearly, Now}^{\alpha} \text{ or Mut}^{\alpha} U}{\text{consume } \propto :: T; ()} \quad \frac{T \text{ is Int, End}^{\alpha} \text{ or Share}^{\alpha} U}{\text{move } \propto :: T; \text{Ur } T} \quad \text{linearly } \propto :: \text{Linearly } \rightarrow \text{Ur } T; \text{Ur } T
\end{array}$$

$$\begin{array}{c}
\frac{T \text{ is Linearly, Ref } U, \text{Now}^\alpha \text{ or Mut}^\alpha U}{\text{withLinearly} \propto :: T; (\text{Linearly}, T)} \quad \text{newRef} \propto :: T; \text{Ref } T \quad \text{freeRef} \propto :: \text{Ref } T; T \\
\\
\text{newLifetime} \propto :: (\forall \iota. \text{Now}^{\text{al } \iota} \multimap T); T \quad \frac{\alpha \leq \text{al } \iota}{\text{endLifetime} \propto :: \text{Now}^{\text{al } \iota}; \text{End}^\alpha} \\
\\
\frac{\alpha \leq \beta \quad T \preccurlyeq U}{\text{borrow} \propto :: \text{Linearly}, T; (\text{Mut}^\alpha T, \text{Lend}^\beta U)} \quad \frac{T \preccurlyeq U}{\text{share} \propto :: \text{Mut}^\alpha T; \text{Ur} (\text{Share}^\alpha U)} \\
\\
\frac{T \text{ is Int, End}^\beta \text{ or Share}^\beta U \quad T \preccurlyeq T'}{\text{copy} \propto :: \text{Mut}^\alpha T; T'} \quad \frac{T \text{ is Int, End}^\beta \text{ or Share}^\beta U}{\text{copy} \propto :: \text{Share}^\alpha T; T} \\
\\
\frac{\alpha' \leq \alpha \wedge \beta}{\text{joinMut} \propto :: \text{Mut}^\alpha (\text{Mut}^\beta T); \text{Mut}^{\alpha'} T} \quad \frac{\alpha' \leq \alpha \wedge \beta \quad T \preccurlyeq U}{\text{joinMut} \propto :: \text{Share}^\alpha (\text{Mut}^\beta T); \text{Share}^{\alpha'} U} \\
\\
\text{reclaim} :: \text{Lend}^\alpha T, \text{End}^\alpha; T \quad \text{execBO} \propto :: \text{Now}^\alpha, \text{BO}^\alpha T; (\text{Now}^\alpha, T) \\
\\
\text{pure} \propto :: T; \text{BO}^\alpha T \quad (\gg) \propto :: \text{BO}^\alpha T, T \multimap \text{BO}^\alpha U; \text{BO}^\alpha U \\
\\
\text{sexecBO} \propto :: \text{Now}^\alpha, \text{BO}^{\alpha \wedge \beta} U; \text{BO}^\beta (\text{Now}^\alpha, U) \quad \text{parBO} \propto :: \text{BO}^\alpha T, \text{BO}^\alpha U; \text{BO}^\alpha (T, U) \\
\\
\frac{\beta \leq \alpha}{\text{deref} \propto :: B^\alpha (\text{Ref } T); \text{BO}^\beta (B^\alpha T)} \quad \frac{\beta \leq \alpha}{\text{updateRef} \propto :: T \multimap \text{BO}^\beta (U, T), \text{Mut}^\alpha (\text{Ref } T); \text{BO}^\beta (U, \text{Mut}^\alpha (\text{Ref } T))} \\
\\
\text{execBO}^\circ \propto :: \text{Done}^\rho T; (\text{Now}^{\text{al } [\rho]}, T) \quad \frac{\alpha \leq \text{al } [\rho]}{(\gg^\circ) \propto :: \text{Done}^\rho T, T \multimap \text{BO}^{\text{al } [\rho]} U; \text{BO}^\alpha U} \\
\\
\frac{\forall i. P' \subseteq P_i}{\text{NOW}_{H;P}^{\rho;0}, \text{now}_{::1}^{P''} \text{Now}^\alpha, \text{bo}_{::1}^{P'} \text{BO} \wedge \overline{\text{al } [\rho]} \wedge \alpha \vdash \text{sexecBO}^+ \text{now } \text{bo} \propto \text{sexecBO}_{H}^+ \text{now } \text{bo} :: \text{Done}^{\tilde{P}} T} \\
\\
\frac{\text{NOW}_{H;P}^{\rho;1}, \text{NOW}_{H';P'}^{\rho';1}, \text{res}_{::1}^{P''} \text{Done}^{\overline{\rho;0}, \rho';0} T \vdash \text{sexecBO}^\circ \text{res} \propto \text{sexecBO}_{H,H'}^\circ \text{res} :: \text{Done}^{\tilde{P}} T}{\forall i. P', P'' \subseteq P_i} \\
\\
\frac{\text{NOW}_{H;P}^{\rho;2}, \text{res}_0 ::_1^{P'} \text{Done}^{\overline{\rho;0}, \rho';0} T, \text{res}_1 ::_1^{P''} \text{Done}^{\overline{\rho;1}, \rho';1} U \vdash \text{parBO}^\circ \text{res}_0 \text{res}_1 \propto \text{parBO}_H^\circ \text{res}_0 \text{res}_1 :: \text{Done}^{\tilde{P}} (T, U)}{\wedge \overline{\text{al } [\rho]} \leq \alpha} \\
\\
\frac{\text{NOW}_{H;P}^{\rho;0}, \text{ref}_{::1}^{P'} B^\alpha (\text{Ref } T) \vdash \text{deref}^\circ \text{ref} \propto \text{deref}_H^\circ \text{ref} :: \text{Done}^{\tilde{P}} (B^\alpha T)}{\wedge \overline{\text{al } [\rho]} \leq \alpha} \\
\\
\frac{\text{NOW}_{H;P}^{\rho;0}, \text{ko}_{::1}^{P''} T \multimap \text{BO}^{\overline{\text{al } [\rho]}} (U, T), \text{ref}_{::1}^{P'} \text{Mut}^\alpha (\text{Ref } T) \vdash \text{updateRef}^+ \text{ko } \text{ref} \propto \text{updateRef}_H^+ \text{ko } \text{ref} :: \text{Done}^{\tilde{P}} (U, \text{Mut}^\alpha (\text{Ref } T))}{\forall i. (\wedge \overline{\text{al } [\rho]} \leq \alpha_i) \quad \forall i. (\beta \leq \alpha_i) \quad \forall i. (T'_i \preccurlyeq \text{Ref } T) \quad \forall i. (\text{Ref } T \preccurlyeq T'_i)} \\
\\
\frac{x = \text{Ref } \ell \propto \text{Ref } y, \ell \mapsto y, \text{MUT}^{\tilde{P}}, \text{DEPO}^{\tilde{P}}, \overline{\text{LIFETIME}_\alpha^{[p]}}, \text{BORROW}_{x;T}^{\tilde{P}}, \text{res}_{::1}^P \text{Done}^{\tilde{P}} (U, T) \vdash \text{updateRef}_\ell^\circ \text{res} \propto \text{updateRef}_{\tilde{P}}^\circ \text{res} :: \text{Done}^{\tilde{P}} (U, \text{Mut}^\beta (\text{Ref } T))}{\forall i. (\wedge \overline{\text{al } [\rho]} \leq \alpha_i) \quad \forall i. (\beta \leq \alpha_i) \quad \forall i. (T'_i \preccurlyeq \text{Ref } T) \quad \forall i. (\text{Ref } T \preccurlyeq T'_i)} \\
\\
\frac{x = \text{Ref } \ell \propto \text{Ref } y, \ell \mapsto y, \text{MUT}^{\tilde{P}}, \text{DEPO}^{\tilde{P}}, \overline{\text{LIFETIME}_\alpha^{[p]}}, \text{BORROW}_{x;T}^{\tilde{P}}, \text{NOW}_{H;P}^{\rho;0}, \text{res}_{::1}^{P'} (U, T) \vdash \text{updateRef}_\ell^\circ \text{pr} \propto \text{updateRef}_{\tilde{P};H}^\circ \text{pr} :: \text{Done}^{\tilde{P}} (U, \text{Mut}^\beta (\text{Ref } T))}{}
\end{array}$$

$$\begin{array}{c}
\frac{\beta \leq \bigwedge_i \alpha_i \quad T \preceq U \quad U \preceq T}{\text{MUT}^{\bar{p}}, \text{LIFETIME}_{\alpha}^{[p]}, \text{BORROW}_{x;T}^{\bar{p}}, x = \hat{v} \propto \hat{v} \vdash \hat{v} \propto \text{Mut}^{\bar{p}} \hat{v} :: \text{Mut}^{\beta} U} \quad \frac{\beta \leq \bigwedge_i \alpha_i \quad T \preceq U}{\text{SHARE}^{\bar{p}}, \text{LIFETIME}_{\alpha}^{[p]}, \text{BORROW}_{x;T}^{\bar{p}}, x = \hat{v} \propto \hat{v} \vdash \hat{v} \propto \text{Share}^{\bar{p}} \hat{v} :: \text{Share}^{\beta} U} \\
\\
\frac{\forall i. \hat{\Gamma}_i \vdash x_i \propto \text{Share } x_i :: \text{Share}^{\alpha} (T_{j,i}[\overline{U/X}]) \quad \text{data } F \bar{X} \text{ where } C :: \overline{T \rightarrow_{\pi} F \bar{X}}}{\sum_i \pi_{j,i} \hat{\Gamma}_i \vdash C_j \bar{x} \propto \text{Share } C_j \bar{x} :: \text{Share}^{\alpha} (F \bar{U})} \\
\\
\frac{\hat{\Gamma} \vdash x \propto \text{Share } x :: \text{Share}^{\alpha} T}{\ell \xrightarrow{\hat{p}} x + \hat{\Gamma} \vdash \text{Ref } \ell \propto \text{Share } (\text{Ref } x) :: \text{Share}^{\alpha} (\text{Ref } T)} \\
\\
\frac{x ::_{\pi}^P T, \hat{\Gamma} \vdash \hat{t} \propto i :: \hat{U}}{\text{VAR}_{\forall x.T}^x, x ::_{\pi}^P T, \hat{\Gamma} \vdash \hat{t} \propto i :: \hat{U}} \quad \frac{T \preceq U}{\text{SKEL}_T^x, x \propto x :: \text{Skel } U} \quad \frac{T \preceq U}{\text{SKEL}_T^x, x = \hat{v} \propto \hat{v} \vdash \hat{v} \propto \hat{v} :: \text{Skel } U} \\
\\
x ::_{\omega}^P T \vdash x \propto x :: \text{Skel } T \quad x ::_{\omega}^P T, x = \hat{v} \propto \hat{v} \vdash \hat{v} \propto \hat{v} :: \text{Skel } T \\
\\
\frac{\hat{\Gamma} \vdash \hat{t} \propto i :: \forall x. \text{Skel } T}{\hat{\Gamma} \vdash \hat{t} \propto i :: \text{Skel } (\forall x. T)} \quad \frac{\forall i. \hat{\Gamma}_i \vdash x_i \propto x_i :: \text{Skel } (T_{j,i}[\overline{U/X}]) \quad \text{data } F \bar{X} \text{ where } C :: \overline{T \rightarrow_{\pi} F \bar{X}}}{\sum_i \pi_{j,i} \hat{\Gamma}_i \vdash C_j \bar{x} \propto C_j \bar{x} :: \text{Skel } (F \bar{U})} \\
\\
\frac{T \text{ is } U \rightarrow_{\pi} U', \text{Linearly, Ref } U, \text{Now}^{\alpha}, \text{Lend}^{\alpha} U, \text{ or } \text{BO}^{\alpha} U}{\vdash \hat{v} \propto \hat{v} :: \text{Skel } T} \\
\\
\frac{\hat{\Gamma} \vdash \hat{v} \propto \hat{v} :: \text{Skel } T}{\hat{\Gamma} \vdash \hat{v} \propto \text{Mut}^{\bar{p}} \hat{v} :: \text{Skel } (\text{Mut}^{\alpha} T)} \quad \frac{\hat{\Gamma} \vdash \hat{v} \propto \hat{v} :: \text{Skel } T}{\hat{\Gamma} \vdash \hat{v} \propto \text{Share } \hat{v} :: \text{Skel } (\text{Share}^{\alpha} T)} \\
\\
\frac{\bar{q} \neq \emptyset \quad \hat{\Gamma} \vdash_{\hat{H}}^{\bar{q}, \bar{p}} x :: T \quad \hat{\Delta} \vdash \hat{t} \propto i :: \hat{U}}{\text{DEPO}^{\bar{p}}, \text{BORROW}_{x;T;\bar{q}}^{\bar{p}} + \text{RELEND}_{\hat{H}}^{\bar{q}, \bar{p}} + \hat{\Gamma} + \hat{\Delta} \vdash \hat{t} \propto i :: \hat{U}} \\
\\
\frac{x = \hat{v} \vdash_{\emptyset}^{\bar{p}} \hat{t} \rightsquigarrow \hat{t}}{\hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{v} \rightsquigarrow \hat{w} \quad x = \hat{v} + y = \hat{w} + \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} x \rightsquigarrow y} \quad \frac{\hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}, \bar{0}} y \rightsquigarrow z}{\hat{\Gamma} \vdash_{\hat{p} \leftarrow y, \hat{H}}^{\bar{p}} \hat{B} (\text{Ref } x) \rightsquigarrow \hat{B} (\text{Ref } z)} \\
\\
\frac{\forall i. p_i \notin \text{dom } \hat{H} \quad \hat{H} \neq \emptyset \quad \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}, \bar{0}} x \rightsquigarrow y}{\hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{B} (\text{Ref } x) \rightsquigarrow \hat{B} (\text{Ref } y)} \quad \frac{\hat{H} \neq \emptyset \quad \forall i. \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}, \bar{i}} x_i \rightsquigarrow y_i}{\hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{B} (C \bar{x}) \rightsquigarrow \hat{B} (C \bar{y})} \\
\\
\frac{\hat{\Delta} \preceq \hat{\Gamma} \quad \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{t} \propto i :: \hat{T}}{\hat{\Delta} \vdash_{\hat{H}}^{\bar{p}} \hat{t} \propto i :: \hat{T}} \quad \frac{x \text{ is fresh in } \hat{\Gamma} \quad \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{t} \propto i :: T}{\hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{t} \propto i :: \forall x. T} \\
\\
\frac{H \neq \emptyset \quad \forall i. \hat{\Gamma}_i \vdash_{\hat{H}_i}^{\bar{p}, \bar{i}} x_i \propto x_i :: T_{j,i}[\overline{U/X}] \quad \text{data } F \bar{X} \text{ where } C :: \overline{T \rightarrow_{\pi} F \bar{X}}}{\sum_i \hat{\Gamma}_i \vdash_{\hat{H}}^{\bar{p}} C_j \bar{x} \propto C_j \bar{x} :: F \bar{U}} \\
\\
\frac{\beta \leq \bigwedge_i \alpha_i \quad T \preceq U \quad U \preceq T \quad \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{v} \rightsquigarrow \hat{w}}{\text{MUT}^{\bar{q}}, \text{LIFETIME}_{\alpha}^{[q]}, \text{BORROW}_{x;T;\bar{p}}^{\bar{q}}, x = \hat{v} \propto \hat{w}, \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{v} \propto \text{Mut}^{\bar{q}} \hat{v} :: \text{Mut}^{\beta} U} \\
\\
\frac{\alpha \leq \text{al } \iota \quad \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{t} \propto i :: \text{Skel } T}{\text{END}_{\hat{H}}^{\iota}, \hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \hat{t} \propto \text{Mut}^{\bar{q}} i :: \text{Mut}^{\alpha} T} \quad \frac{\hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}, \bar{0}} x \rightsquigarrow y}{\hat{\Gamma} \vdash_{\hat{H}}^{\bar{p}} \text{Ref } \ell \propto \text{Ref } x :: \text{Skel } (\text{Ref } T)} \\
\\
\frac{H \neq \emptyset \quad \forall i. \hat{\Gamma}_i \vdash_{\hat{H}_i}^{\bar{p}, \bar{i}} x_i :: \text{Skel } (T_{j,i}[\overline{U/X}]) \quad \text{data } F \bar{X} \text{ where } C :: \overline{T \rightarrow_{\pi} F \bar{X}}}{\sum_i \hat{\Gamma}_i \vdash_{\hat{H}}^{\bar{p}} C_j \bar{x} \propto C_j \bar{x} :: \text{Skel } (F \bar{U})}
\end{array}$$

C Metatheory

We present the omitted details of our metatheory introduced in §5.5.

C.1 Strong Confluence of Denotational Operational Semantics

PROOF OF THEOREM 5.1. We say that the target variable of a (derivation of a) reduction $\dot{E}; x \rightarrow \dot{E}'; x$ is y if, other than **DRED-KTX**, the reduction applies a rule whose conclusion is of the form $\dot{E}; y \rightarrow \dot{E}'; y$.

Assume that the target variables of $\dot{G} \rightarrow \dot{G}_1$ and $\dot{G} \rightarrow \dot{G}_2$ are x and y , respectively. If $x = y$, then $\dot{G}_1 = \dot{G}_2$ holds, which contradicts the assumption. So we have $x \neq y$. By straightforward case analysis, we can derive the reductions $\dot{G}_1 \rightarrow \dot{G}_+$ and $\dot{G}_2 \rightarrow \dot{G}_+$ for some $\dot{G}_+$, choosing y as the target variable of the former reduction and x for the latter. A notable case is where one reduction executes **reclaim DRED-RECLAIM**; the confluence holds in this case, because the structure of restoration by history Fig. 36c does not get affected by the reduction of any variable. $\square$

C.2 Basics of the Association System

Technically, the association judgment does not have a rule for weakening the resulting type. Instead, it is carefully designed so that it is closed under subtyping:

LEMMA C.1 (α IS CLOSED UNDER SUBTYPING). *If $\dot{T} \preccurlyeq \dot{U}$, then $\dot{\Gamma} \vdash \dot{t} \propto \dot{t} :: \dot{T}$ implies $\dot{\Gamma} \vdash \dot{t} \propto \dot{t} :: \dot{U}$.*

PROOF. By straightforward induction over the derivation of subtyping and association. $\square$

For Theorem 5.2, we introduce the following lemma:

LEMMA C.2. *If a term t satisfies $\Gamma \vdash t :: T$, then we have $\Gamma \vdash t \propto t :: T$.*

PROOF. By straightforward induction over the typing derivation. $\square$

PROOF OF THEOREM 5.2. Immediately from Lemma C.2. $\square$

PROOF OF THEOREM 5.3. Straightforward by definition. $\square$

C.3 Progress and Bisimulation

PROOF SKETCH FOR CONJECTURE 5.4. By careful case analysis. In particular, we maintain an invariant on histories to ensure progress for lender reclamation **DRED-RECLAIM** in denotational operational semantics. $\square$

PROOF SKETCH FOR CONJECTURE 5.5. By careful case analysis. There are a few tricky factors. First, the variable that owns each local or global deposit for a borrow can drastically change, especially when a lifetime ends by **endLifetime**; still, the variable always moves to either an ancestor or new children (possibly splitting the deposit), which is crucial for maintaining the disjointness invariant for **parBO**. Also, when copying a value by unrestrictedness or as a skeleton from a variable polymorphically typed by **ASSOC- $\forall$** , we re-type the variable to make it free of **ASSOC- $\forall$** , polymorphically re-typing the variables that the variable depends on. $\square$